

Contenidos Introducción a Club:

Electrónica Básica, Programación y Arduino

Autores: Gaspar Fábrega R.
Benjamín Espinoza H.
Paula Carrion .

Resumen

En esta versión del **Club de Robótica Mustakis** aprenderemos todo lo necesario para el segundo semestre, donde podremos especializarnos en tres ramas de club!

Los contenidos de este taller semestral incluyen:

- Electrónica Básica
- Bases de la Programación

Índice

1. Electrónica Básica	3		
1.1. ¿Qué es la electrónica?	3		
1.1.1. ¿Qué son los electrones (e^-)?	3		
1.1.2. ¿Que son los metales?	4		
1.1.3. ¿Que es la electricidad?	4		
1.1.4. Carga	4		
1.1.5. Corriente	4		
1.1.6. Resistencia	5		
1.1.7. Voltaje	5		
1.1.8. Potencia	5		
1.2. Componentes Electrónicos Básicos	6		
1.2.1. Protoboard	6		
1.2.2. Fuentes de Poder	6		
1.2.3. Interruptores	7		
1.2.4. Resistencias	8		
1.2.5. Diodos	9		
1.2.6. Desafío I:	10		
1.2.7. Capacitores	10		
1.2.8. Transistores	10		
1.2.9. Buzzer Piezoelectrico	11		
1.3. Desafío:	11		
1.4. EXTRA: Circuitos Integrados	12		
1.5. Desafío Final:	13		
2. Bases de la Programación:	14		
2.1. ¿Qué es un lenguaje de programación?	14		
2.2. Estructura General	14		
		2.2.1. Entorno Global	14
		2.2.2. void setup()	14
		2.2.3. void loop()	14
		2.3. Variables	14
		2.3.1. Variables globales	14
		2.3.2. Variables locales	14
		2.3.3. Tipos de variables	14
		2.3.4. Arreglos (listas)	15
		2.4. Lógica proposicional	15
		2.4.1. Comparador lógico &&	15
		2.4.2. Comparador lógico	15
		2.5. Aritmética	15
		2.5.1. Comparación Aritmética	15
		2.5.2. Operadores aritméticos	15
		2.6. Consola Serial	16
		2.7. Estructuras condicionales	16
		2.7.1. if	16
		2.7.2. while	17
		2.7.3. for	17
		2.8. Funciones	17
		2.8.1. Funciones básicas	18
		2.8.2. Funciones con entrada(Input)	18
		2.8.3. Funciones con salida(Output)	18
		2.8.4. Funciones Input-Output	19
		2.9. Desafíos	19

3. Circuitos Programables:	20
3.1. Microcontroladores:	20
3.2. Arduino:	20
3.3. Arduino Pro Micro:	21
3.4. Energía:	22
3.5. Señales Digitales:	22
3.5.1. Lectura de señales digitales:	22
3.5.2. Emisión de señales digitales:	23
3.6. Señales Análogas:	24
3.6.1. Lectura de señales análogas:	24
3.6.2. Escritura de señales análogas: . . .	25
3.6.3. Otros usos de PWM	25
3.6.4. buzzer piezoeléctrico pasivo	25
4. Análisis y diseño de algoritmos:	26
4.1. ¿Qué son los algoritmos?	26
4.2. Comunicación Serial	26
4.2.1. Serial.begin()	26
4.2.2. Serial.print() y Serial.println() . .	26
4.2.3. Serial.available()	26
4.2.4. Serial.read()	27
4.2.5. Strings y Serial.readStringUntil('\n')	27
4.3. Debugging	27
4.4. Ejercicios	27
4.4.1. Preséntate!!	27
4.4.2. Círculos	28
4.5. Patrones	28
4.6. Desafíos	29
4.6.1. Triángulos	29
4.6.2. Secuencia de Collatz	29
4.6.3. No múltiplos	30
4.6.4. Alzas del dólar	30
4.6.5. Promoción con descuento (Difícil)	30
4.6.6. Intersección de círculos(Difícil) . .	30
5. Laboratorio de electrónica y programación:	32
5.1. Ingeniería inversa:	32
6. Laboratorio extra:	33
6.1. Calculadora:	33
6.1.1. Pantalla OLED	33
6.1.2. Matriz de botones 4x4	34

1. Electrónica Básica

1.1. ¿Qué es la electrónica?

“Campo de la ciencia que estudia el flujo y control de electrones (electricidad) y sus aplicaciones.”

1.1.1. ¿Qué son los electrones (e^-)?

“Partícula subatómica con carga negativa y pequeña masa. Junto al protón (p^+) y al neutrón (n^0), es uno de los tres componentes del átomo”

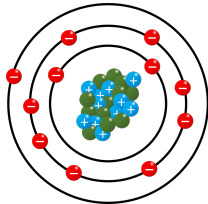


Figura 1: Diagrama básico de un átomo

Descubierto en 1881 por J. J. Thompson, el electrón es la primera partícula subatómica en ser descubierta y clasificada como tal. El descubrimiento viene del uso de un **tubo de rayos catódicos**, donde a partir de un cañón de electrones (un dispositivo que emite electrones en línea recta) se pudo observar la emisión de un rayo de partículas que interactuaban con cargas eléctricas.

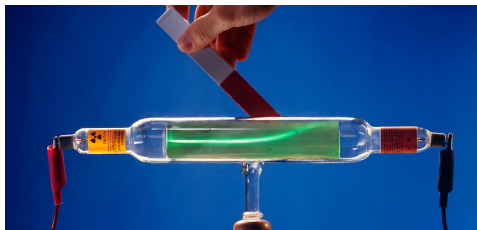


Figura 2: Tubo de rayos catódicos utilizado por Thompson

Este descubrimiento dio paso a una revolución en la física culminando con el establecimiento del **modelo estándar de partículas**, donde se clasifican todas las partículas elementales que componen nuestro universo.

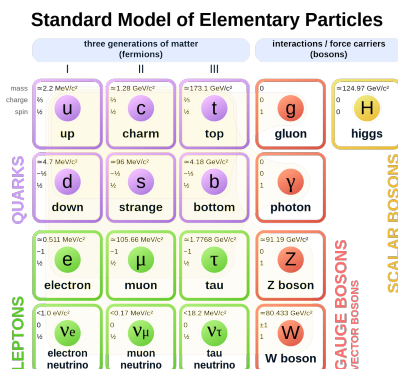


Figura 3: Todas las partículas actualmente conocidas.

Y la **mecánica cuántica**, el modelo actual que explica como estas partículas elementales interactúan entre sí, formando átomos. Según el modelo cuántico, el átomo está compuesto de protones y neutrones (a su vez, formados de partículas elementales llamadas *quarks*) agrupados en un núcleo, con una “nube” de electrones a su alrededor. Esta “nube” corresponde una región del espacio al rededor del núcleo donde el electrón tiene **altas probabilidades de estar**.

WARNING: Estos conceptos son difíciles de explicar, entender y demostrar. Nos enfocaremos solamente en mostrar diagramas y explicar de manera superficial estos temas.

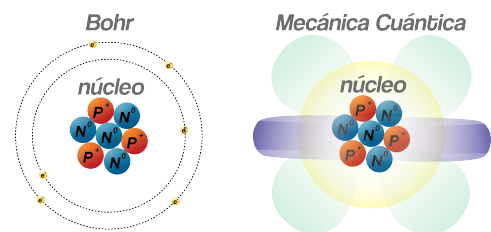


Figura 4: Diagrama de Bohr, con orbitales planetarios vs el diagrama de un átomo según el modelo cuántico.

Estas “nubes” de electrones (llamadas orbitales) toman distintas formas dependiendo de la energía que tienen y se van superponiendo unas con otras, rellenando distintos niveles. Los orbitales electrónicos pueden estar en tres estados:

- **Vacíos:** No se encuentran electrones en el orbital.
- **Parcialmente llenos:** El orbital está parcialmente lleno, lo que lleva a dos situaciones:
 - El átomo busca deshacerse de los electrones de su último orbital, para quedarse con el último orbital completamente lleno.

elementos electropositivos

- El átomo busca capturar electrones para llenar su último orbital.

elementos electronegativos

- **Llenos:** Tiene todos los electrones posibles, este estará en una configuración estable y es muy difícil incluir o extraer electrones.

Este comportamiento da paso a la definición de los **electrones de valencia**, los electrones que se encuentran en el último orbital y que pueden interactuar con otros átomos dando paso al comportamiento eléctrico y químico de un elemento.

1.1.2. ¿Que son los metales?

Para tener una idea clara de el por qué utilizamos metales en electrónica y cómo se transmite la electricidad, explicaremos que son los metales y por que se utilizan.[2] Los metales pueden tener varias definiciones:

- **General:** *Una sustancia con una alta conductividad eléctrica, lustre y maleabilidad.*
- **Química:** *Elementos capaces de perder electrones para formar enlaces metálicos.*
- **Astronomía** *Todos los elementos más pesados que el Litio*

En la tabla periódica de elementos, podemos ver que la gran mayoría de elementos conocidos son metales.

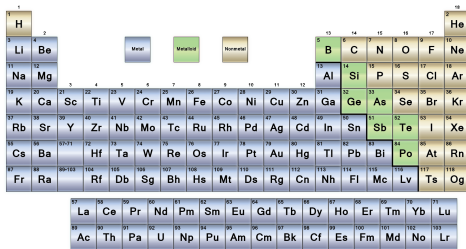


Figura 5: Tabla periódica de elementos. **Metales** en gris, **metaloides** en verde y **no metales** en beige

La razón por la cual los metales son buenos conductores eléctricos viene de la descripción química: **forman enlaces metálicos**. Este tipo de enlace es interesante, ya que los elementos involucrados se liberan de su última capa de electrones, formando un “mar de electrones” capaces de fluir libremente al rededor de los núcleos.

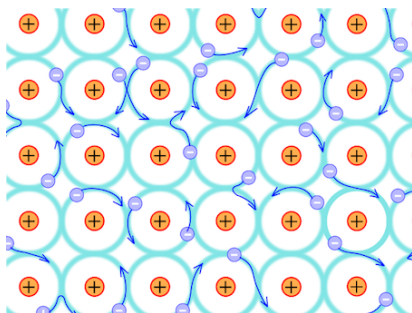


Figura 6: Mar de electrones en el enlace metálico.

¿Sabías qué? La razón por la que los metales forman enlaces metálicos, nuevamente se explica con mecánica cuántica, y puede resumirse en: **su banda de valencia interseca la banda de conducción**, esto significa que la energía necesaria para perder un electrón coincide con la energía necesaria para que el electrón esté en el último orbital.

¿Muchas palabras extrañas?

1.1.3. ¿Que es la electricidad?

La electricidad es un fenómeno complejo[1], que para ser completamente explicado, requiere entender múltiples áreas de la física. De la manera más simplificada, la electricidad es:

“Flujo de carga eléctrica a través de un material.”

Esta carga eléctrica en la gran mayoría de los casos corresponde a **electrones**, pero en algunas situaciones (como en la transmisión de señales de nuestras neuronas o en el agua) corresponde a **iones** (átomos con exceso o falta de electrones).

El comportamiento de la electricidad en un circuito de corriente continua puede explicarse utilizando la **analogía del agua [3]**, con la que iremos explicando los conceptos básicos de la electricidad:

WARNING: la analogía del agua no es 100 % adecuada para explicar todos los fenómenos involucrados en circuitos eléctricos, ya que al ser una simplificación, no toma en cuenta elementos específicos de la electricidad como los campos magnéticos generados

1.1.4. Carga

La carga es una propiedad fundamental del universo, los principales encargados de transmitir carga son el **electrón** (carga negativa) y el **protón** (carga positiva), estas partículas tienen igual **magnitud** (el mismo valor numérico), pero signos opuestos.

La carga de la que hablamos al explicar la electricidad corresponde a **electrones** y se mide en [coulomb]. La carga de un electrón es:

$$Q_{e-} = 1.602 \cdot 10^{-19} \text{ [coulomb]}$$

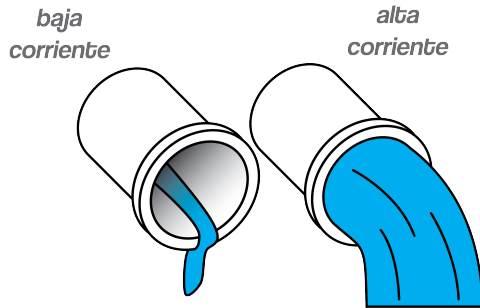
En nuestra analogía, la carga corresponderá a agua que fluye a través de un sistema de tuberías.

1.1.5. Corriente

La corriente es el flujo de carga que se mueve a través de un material. Esto se mide en unidades de carga por unidad de tiempo.

$$\text{corriente} = \frac{\text{carga}}{\text{tiempo}} \frac{[\text{coulomb}]}{[\text{segundo}]}$$

En nuestra analogía del agua la corriente corresponde a la cantidad de agua que fluye por una tubería.

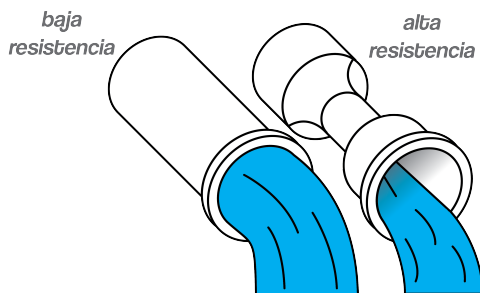


1.1.6. Resistencia

La resistencia corresponde a la oposición que presenta un material al paso de la corriente (el movimiento de electrones a través de este). Esta resistencia dependerá de propiedades del material, es decir su *resistividad* (elementos que lo componen, tipos de enlaces, mezclas de compuestos existentes, etc) y sus dimensiones:

$$resistencia = resistividad \cdot \frac{largo}{area}$$

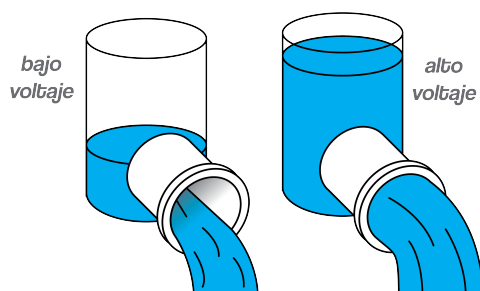
En la analogía del agua, la resistencia en un circuito corresponde a una constricción (disminución de tamaño) en las tuberías, ofreciendo una mayor resistencia al flujo del agua, y disminuyendo la cantidad de agua que puede pasar.



1.1.7. Voltaje

El voltaje se define como la diferencia de potencial eléctrico (energía) entre dos puntos, esta diferencia de potencial, es la que empuja a los electrones, desde el punto de **mayor** potencial al de *menor*.

En la analogía del agua, el voltaje es equivalente a la presión generada por una torre de agua o por una bomba, donde el agua fluirá a través de las tuberías desde el punto de **mayor** presión hacia un punto de *menor* presión.



Mientras mayor presión exista, mayor será el flujo de agua, lo mismo es equivalente para circuitos de corriente

continua, mientras mayor sea el voltaje, mayor será la corriente generada.

Para calcular el voltaje entre dos puntos, se utiliza la **ley de ohm**:

$$voltaje = corriente \cdot resistencia$$

1.1.8. Potencia

La potencia, corresponde a la cantidad de energía que el circuito consume en cada segundo, esta energía depende de el flujo de electrones y de el voltaje que están pasando por el circuito.

$$potencia = voltaje \cdot corriente$$

Mientras mayor sea el voltaje o la corriente, mayor es la energía utilizada.

1.2. Componentes Electrónicos Básicos

1.2.1. Protoboard

Una **protoboard** o **prototyping board** (*placa de pruebas*) es una herramienta que nos permite conectar componentes para generar circuitos de manera rápida, corregir errores y probar múltiples componentes sin necesidad de soldar. Consiste de una placa plástica con múltiples agujeros ordenados en filas y columnas, conectadas eléctricamente en la base. El siguiente diagrama explica cómo están conectadas las filas y columnas de la placa:

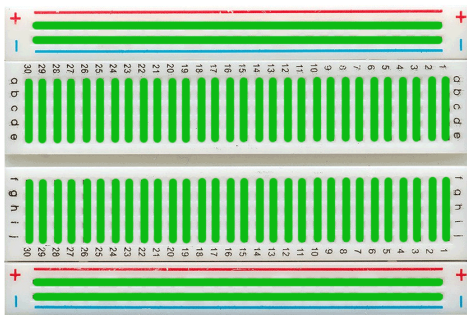


Figura 7: Diagrama de las conexiones en una protoboard.

- **Columnas:** Enumeradas desde el 1 en adelante. Cada columna contiene dos series de agujeros conectados eléctricamente entre sí, limitados por el centro de la placa.
- **Filas Etiquetadas** con las letras 'a' a la 'j'. Sirven para enumerar los agujeros que están conectados entre sí en una columna
 - 'a' a la 'e' en la mitad superior
 - 'f' a la 'j' en la mitad inferior
- **Energía:** Dos filas en la parte superior y dos en la inferior, con símbolos + y -, sirven para energizar los componentes del circuito desde una fuente en común.

1.2.2. Fuentes de Poder

Una fuente de poder va a ser cualquier dispositivo que sea capaz de entregar energía eléctrica para alimentar nuestros circuitos, en nuestro caso utilizaremos baterías, pero existen múltiples maneras de obtener energía eléctrica. En nuestra analogía del agua, tendremos que una fuente de poder actúa como una bomba que impulsa el agua.

Baterías: Una batería es un dispositivo electroquímico que almacena y es capaz de entregar energía eléctrica, a partir de una reacción química. Puede dividirse en tres partes:

- **Ánodo:** el terminal negativo, desde donde provienen los electrones que se liberan de la reacción química que ocurre al interior.

- **Electrolito:** corresponde a una interfaz que separa física y eléctricamente el ánodo del cátodo, pero permite que fluyan los iones necesarios para completar la reacción que impulsa los electrones. Al separar eléctricamente los dos terminales, es necesaria una conexión externa para que la reacción química ocurra y comience a liberarse energía eléctrica.
- **Cátodo:** corresponde al terminal positivo, donde se reciben los electrones al cerrar el circuito.

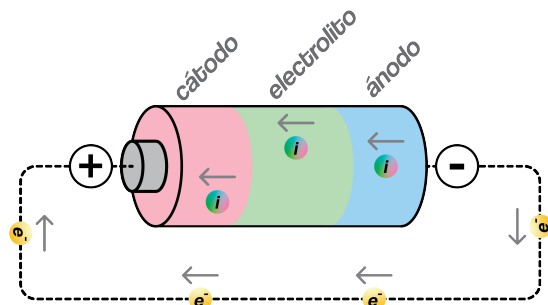


Figura 8: Partes de una batería.

- **Baterías alcalinas:** Las baterías más comunes, baratas y fáciles de encontrar. Derivan su energía de una reacción química entre zinc (Z) y dióxido de manganeso (MnO). Generalmente no son recargables. Una sola celda provee de 1.5 [V], su capacidad y máxima corriente dependen del tamaño, para una AA, la capacidad ronda los 1500 [mAh] y entrega una corriente máxima (antes de perder eficiencia) de 0.5 [A]



- **Baterías de ion-litio:** Las baterías recargables más utilizadas en aplicaciones portátiles. Derivan su energía de una reacción química reversible entre distintos compuestos que contienen litio (Li), donde este elemento se mueve desde el ánodo (-) al cátodo (+) en el interior de la batería, y los electrones se mueven desde el ánodo (-) al cátodo (+) a través del circuito. Una sola celda provee de 3.6 [V].



- **Baterías lipo:** Corresponde a una batería de ion-litio pero donde su electrolito está compuesto de un polímero en vez de un líquido. Una sola celda provee de 3.6 [V].

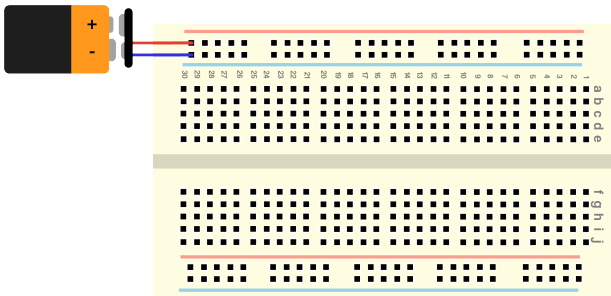


- **Otras baterías:** Existen múltiples otros tipos de baterías, dependiendo de la aplicación para la que se les de uso y los materiales utilizados para almacenar la energía.

Fuente de alimentación: Dispositivo que convierte corriente alterna en corriente continua con un voltaje y corriente específico. Elemento esencial en un laboratorio de electrónica, para poder alimentar distintos circuitos con múltiples requerimientos.



Conectemos: Ahora, procederemos a conectar nuestra fuente de poder (*baterías ******) a nuestra protoboard, siguiendo el siguiente diagrama:



1.2.3. Interruptores

En nuestra analogía del agua, los interruptores funcionan como válvulas, capaces de detener o permitir el flujo de electrones. En general, están compuestas de un elemento mecánico, que al ser activado cierra una serie de contactos (componentes metálicos conectados al circuito), permitiendo el paso de la electricidad.

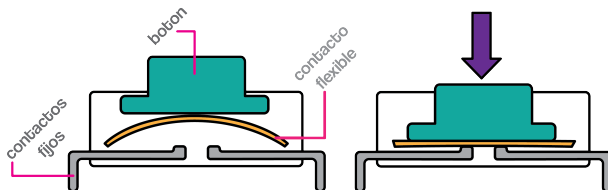


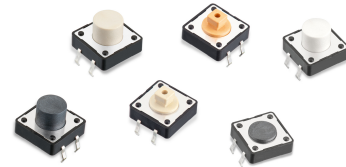
Figura 9: Partes de un Interruptor.

Tipos de interruptores:

- **Basculantes:** Estos interruptores poseen una palanca capaz de cambiar su posición y mantenerla una vez activada, generalmente se utilizan para el encendido o apagado de un dispositivo.



- **Pulsador:** Estos interruptores poseen un botón que debe ser pulsado para cerrar el circuito, generalmente no mantienen su posición y son utilizados para interactuar en tiempo real con un dispositivo.

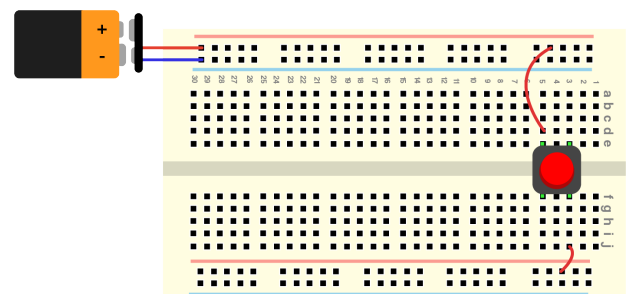


- **Rotativo:** Estos interruptores pueden tener varios estados posibles y son generalmente controlados por la rotación de un eje que cambia los contactos activos.



- **Otros:** Existen una multitud de botones existentes dependiendo de la aplicación en la que se usan y los mecanismos involucrados, **si observas alguno interesante anótalo!**

Conectemos: Conectemos nuestro primer componente en la protoboard, un interruptor, sigamos el siguiente diagrama:



1.2.4. Resistencias

Una resistencia es un componente electrónico que restringe el paso de la corriente, disminuyendo el flujo de electrones entre un punto y otro. En nuestra analogía del agua, una resistencia correspondería a un segmento estrecho en una tubería, que dificulta el paso del agua.

En general, están compuestas de un material con una baja conductividad (generalmente grafito), fabricada con el largo específico para que ofrezca una resistencia predecible.

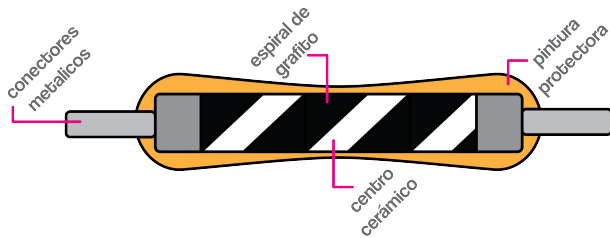


Figura 10: Partes de una Resistencia.

Tipos de resistencias:

- **Resistencias:** Componentes con un valor de resistencia fija, pueden ser fabricados con distintos materiales (*cobertura beige: de grafito, cobertura celeste: de film metálico*) y su forma depende de cómo se unen en el circuito (*through hole: con conectores metálicos largos, surface mounted device: rectangulares*).



- **Potenciómetros:** Los potenciómetros son resistencias variables que pueden ser controladas girando un eje, este eje controla la posición de contactos en su interior que a su vez alargan o acortan el paso de la corriente por un segmento de grafito (material una resistividad alta).



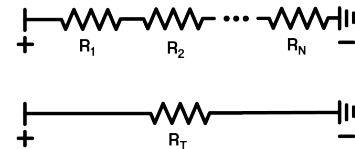
- **Foto-resistencias:** Las foto-resistencias son resistencias que cambian su valor dependiendo de cuanta luz incida sobre ellas. Mientras más luz llega a su superficie, menor será su resistencia, y más corriente dejará pasar.



Reglas para combinar resistencias:

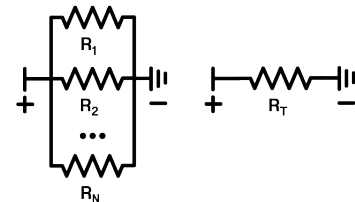
Las resistencias pueden combinarse de distintas maneras para obtener los valores que necesites, dependiendo de si quieres tener una mayor o menor resistencia a las que tienes disponibles.

- **Resistencias en serie:** Si necesitas un valor de resistencia mayor a los valores que tienes disponibles, puedes conectarlas en serie, obteniendo un valor equivalente a la suma de las resistencias utilizadas.



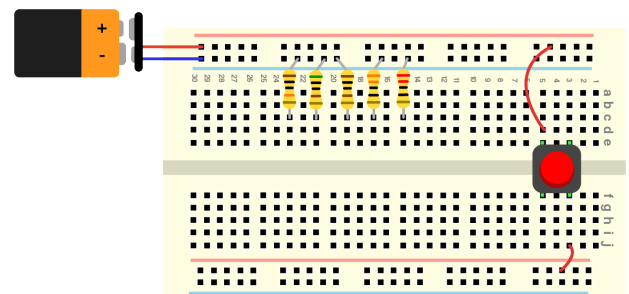
$$R_T = R_1 + R_2 + \dots + R_N$$

- **Resistencias en paralelo:** Si necesitas un valor intermedio entre los valores que tienes disponibles, puedes conectar las resistencias en paralelo, donde el valor equivalente será un valor entre la mayor resistencia y la menor resistencia utilizadas.



$$\frac{1}{R_T} = \frac{1}{R_1} + \frac{1}{R_2} + \dots + \frac{1}{R_N}$$

Conectemos: Conectemos nuestro segundo componente electrónico, tomemos 5 resistencias (220, 330, 1k, 5k y 100k) y posicionémoslas en nuestra protoboard siguiendo el diagrama:

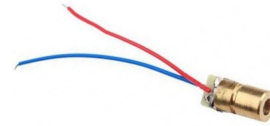


¿Sabías qué? El valor de una resistencia viene catalogado por un código de barras de colores, usa la siguiente página para obtener los colores asociadas a las resistencias solicitadas!

<https://www.digikey.com/en/resources/conversion-calculators/conversion-calculator-resistor-color-code>



- **Diodos LASER:** Diodos que además de sólo permitir el paso de la corriente en una dirección, emiten luz de una longitud de onda específica y en una dirección ordenada.



1.2.5. Diodos

Un diodo es un componente electrónico que solo permite el paso de la corriente en una dirección, ofreciendo resistencia cercana a cero hacia un lado, y resistencia prácticamente infinita hacia el otro. En la analogía del agua, este sería una válvula que solo deja pasar agua en una dirección, tal y como las válvulas antirretorno.

Estos componentes son fabricados con materiales semiconductores, que sólo permiten el paso de la corriente bajo ciertas condiciones.

Los diodos son componentes polarizados, es decir, tienen una dirección en la que deben ser conectados para funcionar de manera correcta (dejando pasar la electricidad), tenemos dos terminales:

- **Cátodo (-):** Va conectado al terminal negativo de la fuente de poder.
- **Ánodo (+):** Va conectado al terminal positivo de la fuente de poder.

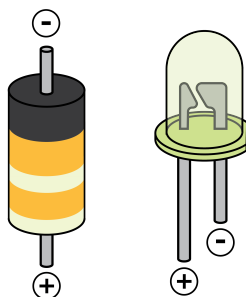
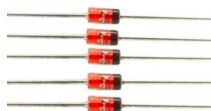


Figura 11: Polaridad de los diodos más comunes.

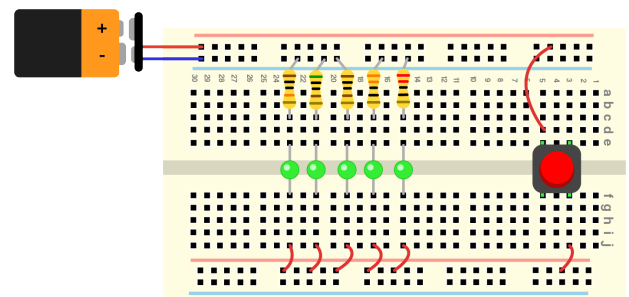
Tipos de diodos:

- **Diodos:** La versión más simple de este componente, cumplen la función para la que fueron diseñados, permitir el paso de la corriente en una sola dirección.



- **Diodos LED:** Diodos que además de sólo permitir el paso de la corriente en una dirección, emiten luz.

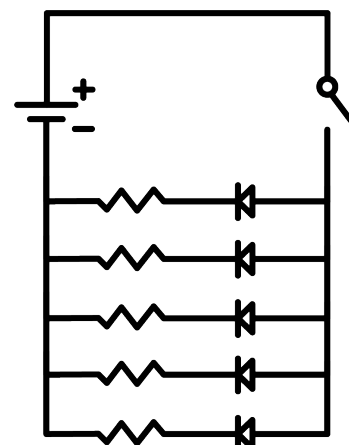
Conectemos: Conectemos nuestro tercer componente, diodos LED. Tomemos LEDs de un mismo color y conectémoslos siguiendo el diagrama a continuación



Este es nuestro primer circuito completo! Observemos el comportamiento de los diodos LED al apretar el interruptor.

- Que crees que sucederá cuando aprietes el interruptor?
- Que observas al apretar el interruptor?
- Prueba intercambiando las resistencias de posición.

A continuación, se muestra el diagrama simplificado del circuito utilizado.



1.2.6. Desafío I:

Conecten las resistencias que acabamos de conocer en serie y en paralelo y realicen predicciones sobre el comportamiento que podremos observar:

- Qué sucederá si conectamos las resistencias de $330\ [\Omega]$ y la de $100\ [k\Omega]$ en serie o en paralelo a un LED? cual configuración será más brillante?
- Qué sucederá si conectamos dos resistencias de $2000\ [\Omega]$ en serie o en paralelo a un LED? cual configuración será más brillante?

1.2.7. Capacitores

Los capacitores con componentes electrónicos que **almacenan energía** en forma de carga eléctrica. Se fabrican separando dos placas de material conductor, mediante una capa de material aislante (material dieléctrico).

Tipos de condensadores:

- **Cerámicos:** Capacitores donde el material dieléctrico es cerámico, dividiendo dos placas de metal delgadas en su interior.



- **Electrolíticos:** Capacitores donde se tiene un cátodo metálico, una capa de material dieléctrico compuesto de óxido, y un ánodo electrolítico (con iones que conducen las cargas). Esto genera que estos condensadores tengan polaridad (una dirección asociada al flujo de la corriente).



El siguiente diagrama muestra la manera de identificar estos dos tipos de condensadores y ver la polaridad de los condensadores electrolíticos.

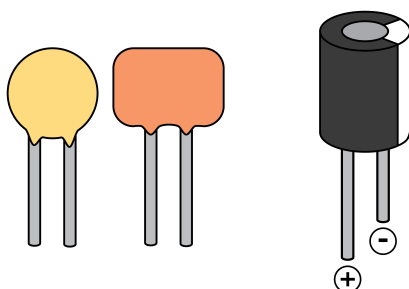
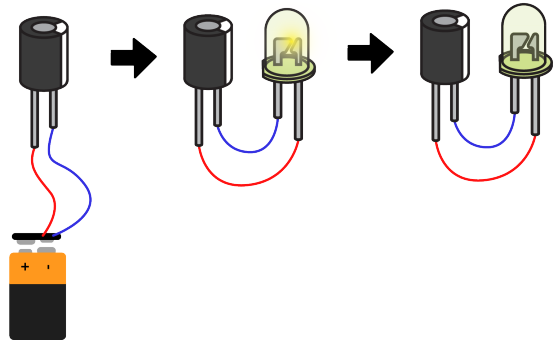


Figura 12: Capacitores cerámicos y polaridad de capacitores electrolíticos.

Podemos observar el comportamiento de estos componentes de manera rápida, cargándolos con la batería (conectando el terminal negativo al ánodo y el terminal positivo al cátodo) por unos segundos, y luego conectando el condensador cargado a un diodo LED, observando cómo se enciende brevemente, consumiendo la energía almacenada entre las placas de metal.



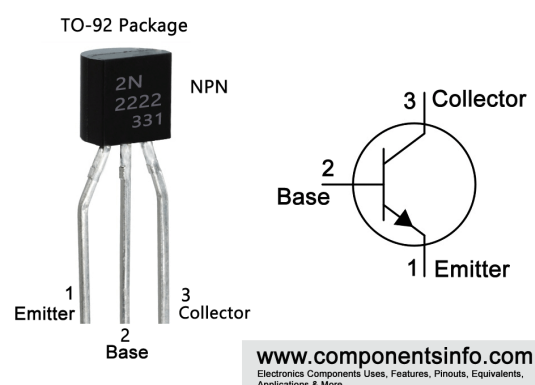
1.2.8. Transistores

Los transistores son componentes electrónicos fabricados con semiconductores, diseñados para **amplificar señales** o para funcionar como **interruptores**. Estos componentes son la base de la electrónica digital, por ello, se consideran la invención más importante del siglo XX.

Estos componentes generalmente poseen tres terminales, donde uno de estos sirve para controlar la corriente que pasa por los otros dos. Tenemos principalmente dos tipos de transistores:

- **Transistor de unión bipolar:** (*BJT, bipolar junction transistor*) Estos transistores poseen tres terminales: **Emisor**, **Base** y **Colector**, cuyas funciones dependen de si son:
 - **NPN:** En este caso, la corriente fluye desde el **Colector** al **Emisor**, y este flujo es posible sólo cuando una corriente pequeña se aplica al terminal **Base**.
 - **PNP:** En este caso, la corriente fluye desde el **Emisor** al **Colector**, y este flujo es posible sólo cuando no hay corriente aplicada en la **Base**.

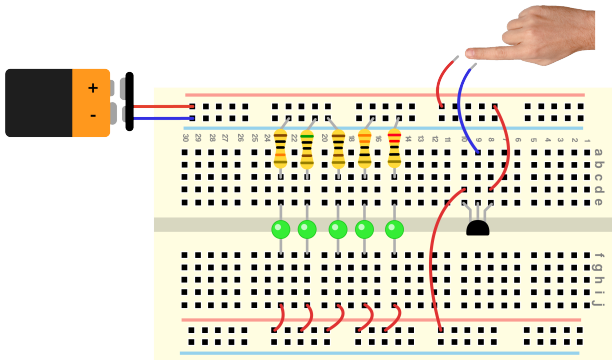
2N2222 Transistor Pinout



- **Transistor de efecto de campo:** (*FET, field effect transistor*) Estos transistores poseen tres terminales: **Puerta**, **Drenador** y **Fuente**. En este caso, la corriente fluye desde la **Fuente** al **Drenador**, y es controlada por el voltaje que se aplica a la **Puerta** (en vez de corriente como en los BJT).

Conectemos: Por ultimo, desconectemos el botón y sus cables asociados, para utilizar un transistor **2n2222**, un transistor NPN, como un interruptor táctil.

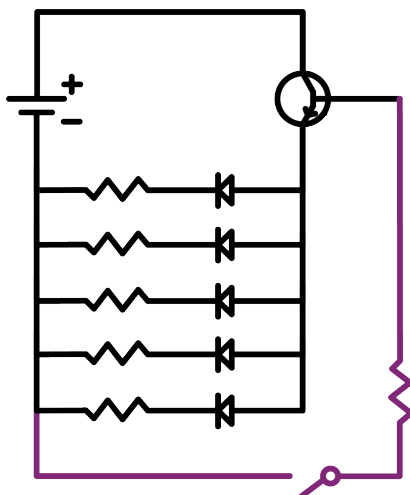
Para ello, usemos la base y el emisor como un interruptor entre las dos filas de alimentación de la proto-board, y conectemos un cable a la columna conectada a la base del transistor.



Ahora, podemos tocar el cable conectado a la base y un cable conectado al terminal positivo de la batería, para activar el transistor y encender las luces LED.

Luego de comprobar el funcionamiento de este transistor, desconectemos todos los componentes para el ejercicio final, procuremos guardar todos los componentes de manera ordenada, para poder utilizarlos más adelante.

A continuación, se muestra el diagrama simplificado del circuito utilizado, donde la parte morada del circuito corresponde a una representación de un dedo como una resistencia muy grande y un interruptor (la acción de tocar los cables).



1.2.9. Buzzer Piezoelectrico

El ultimo componente que aprenderemos a utilizar hoy corresponde a un buzzer piezoelectrico, un pequeño parlante que utiliza materiales cerámicos que son capaces de deformarse bajo la presencia de un campo eléctrico y generar sonido.



Tenemos dos tipos de buzzer:

- **Activo:** Este buzzer es capaz de funcionar sólo con corriente continua, una vez que le entregas un voltaje, un circuito interno se encarga de hacerlo oscilar, generando un tono fijo. **Prueba este buzzer con frecuencias bajas.**
- **Pasivo:** Este buzzer necesita que el voltaje que se le entrega cambie, ya que le movimiento del elemento que genera sonido depende directamente del voltaje aplicado. **Prueba este buzzer con frecuencias altas (mayores a 100 Hz).**

Reemplazemos el LED conectado a nuestro circuito previo por un buzzer piezoelectrico activo y observemos (*escuchemos*) su comportamiento. Aseguremos de que el terminal positivo del buzzer esté conectado al cable proveniente del emisor del transistor y el negativo a tierra.

1.3. Desafío:

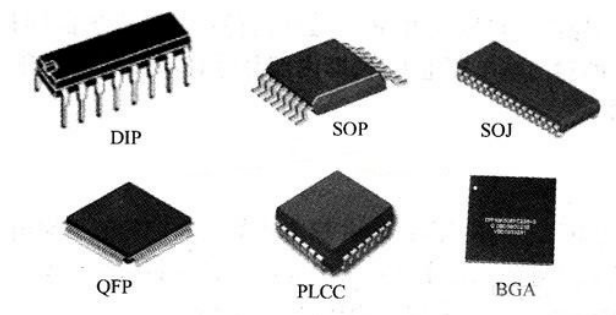
Tomando en cuenta lo aprendido en la clase de hoy, el desafío final corresponde a diagramar los siguientes circuitos:

1. Un botón que controle el encendido de 2 LEDs.
2. Un botón que controle el encendido de un buzzer activo.
3. Dos transistores que controlen el encendido de un LED (deben activarse ambos para encender el LED).
4. Un solo circuito que contenga los tres anteriores

Para cada uno de estos circuitos, debe realizarse un diagrama, el que será validado por un mentor antes de poder pasar a conectarlo en la placa de prototipado.

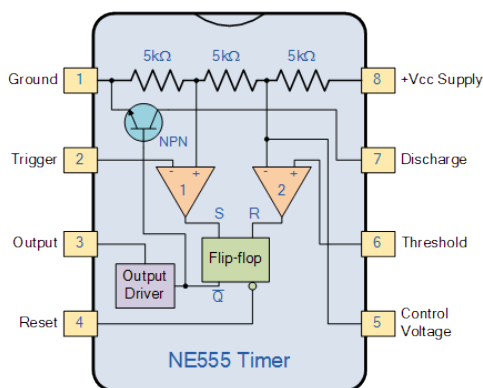
1.4. EXTRA: Circuitos Integrados

Un circuito integrado o “chip” es un circuito electrónico con múltiples componentes empaquetados en un bloque de material semiconductor, al interior de este bloque, se encuentran múltiples transistores miniaturizados y otros componentes electrónicos **integrados** en un circuito.



Existen muchos tipos de circuitos integrados, que van desde **comparadores** (componentes que comparan dos señales e indican cual es la que tiene un mayor voltaje) a **osciladores** (componentes que oscilan con una frecuencia que puede ser controlada) a **sensores** (circuitos que son capaces de traducir señales externas a voltajes) hasta **microcontroladores** y **microcomputadores**.

Temporizador 555: El temporizador 555 es un circuito integrado que permite generar pulsos eléctricos en intervalos continuos, dependiendo de los componentes que se conectan en sus terminales, esto permite prender y apagar una luz o generar sonido con frecuencias específicas. En este taller, lo utilizaremos para conocer cómo los circuitos pueden realizar acciones complejas sin necesidad de programarlo con un computador.



Este circuito integrado posee seis terminales:

1. **GND:** tierra o el terminal negativo, va conectado al terminal negativo de la fuente de poder.
2. **Trigger:** entrada negativa al primer **comparador** del circuito, cuando el voltaje es menor a $0.3 V_{cc}$ se genera un voltaje positivo en la salida (output).

3. **Output:** La señal de salida generada por el el circuito integrado, útil para hacer parpadear luces o generar sonidos.
4. **Reset:** Reinicia el circuito cuando no está recibiendo voltaje, generalmente se mantiene conectado a la batería para evitar reinicios no deseados.
5. **Control:** Puede utilizarse para controlar la frecuencia de salida independiente de las señales conectadas al trigger, generalmente sólo va conectado mediante un capacitor a tierra, para disminuir el ruido.
6. **Threshold:** entrada negativa al segundo **comparador** del circuito, cuando el voltaje es mayor a $0.6 V_{cc}$ se apaga la salida (output).
7. **Discharge:** se utiliza para descargar el circuito utilizado para configurar la frecuencia de salida.
8. **VCC:** Terminal positivo, va conectado el terminal positivo de la fuente de poder, generalmente de entre 5 y 15 V.

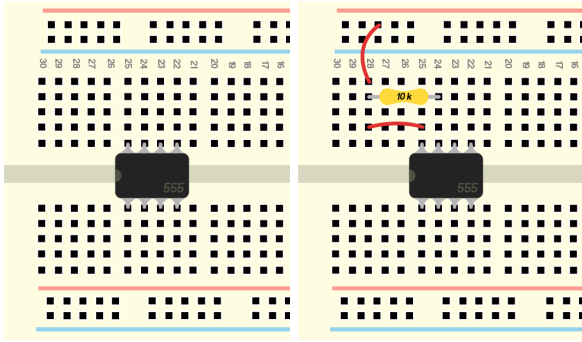
En resumen, utilizaremos los pines de **trigger** y **threshold** para controlar la frecuencia que se genera en el **output**, conectando dos resistencias y un capacitor.

Conectaremos un capacitor al **threshold**, lo que generará que cuando se cargue, el **threshold** se activará, apagando la señal emitida por el 555 (**output**) y activando la descarga del condensador, una vez descargado, se activará el **trigger** y el proceso comenzará nuevamente. El tiempo de carga del capacitor depende de su capacidad de almacenar energía, y la velocidad en la que este se cargará, dependerá del tamaño de la resistencia que está conectada a su terminal positivo.

- Mientras más grande el condensador más tiempo tomará en cargarse y descargarse bajo un mismo voltaje
- Mientras más grande al resistencia, más tiempo tomará en cargarse el condensador, ya que pasará menos corriente hacia este.

Conectemos: Comencemos la ultima experiencia de la clase, siguiendo los siguientes pasos para obtener nuestro circuito final:

1. Como primer paso, conectemos el chip 555 en nuestra protoboard.
2. Realicemos las conexiones necesarias para alimentar el chip con energía desde el terminal positivo de la batería. Conectemos un cable desde el terminal positivo a vcc en el chip 555 y una resistencia de 10k desde el terminal positivo a el terminal de descarga (**discharge**) en el 555.

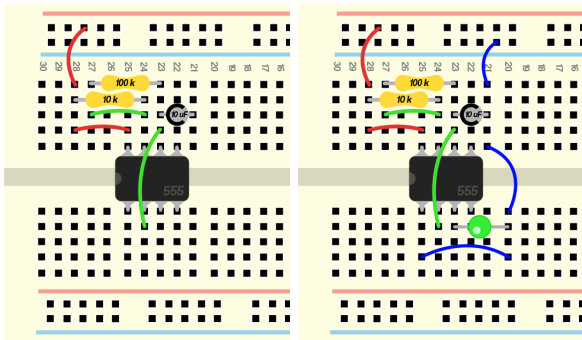


3. A continuación, conectemos la resistencia y el condensador que se cargará y descargará activando la señal de salida:

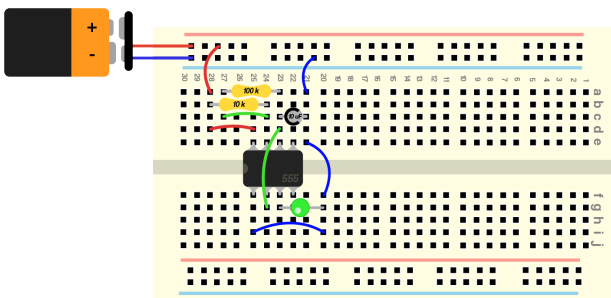
- Conectemos una resistencia de 100k entre el **terminal discharge** y el **terminal threshold** del 555 con atuda de un cable.
- Conectemos un condensador electrolítico de 10 nF entre el **terminal Threshold** y el terminal negativo de la batería.
- Conectemos un cable entre el terminal **discharge** y el terminal **trigger**

4. Por ultimo, realicemos las conexiones a tierra necesarias.

- El terminal **gnd** del chip 555 con ayuda de un cable.
- Un LED desde el terminal **output** a tierra con ayuda de un cable.



Una vez conectados todos los componentes, podemos conectar la batería a las filas de alimentación.



Observemos el comportamiento del diodo LED. Podemos modificar la velocidad con la que el LED parpadea, cambiando los siguientes componentes de acuerdo a la siguiente tabla:

	10 k	100 k
10 μ F	4.8	0.69
4.7 μ F	10	1.5
2.2 μ F	22	3.1
1.0 μ F	48	6.9
0.47 μ F	100	15
0.22 μ F	220	31
0.1 μ F	480	69
0.047 μ F	1000	150

Tabla 1: Frecuencias en Hz que se obtienen al combinar las siguientes resistencias y condensadores, audible en azul y visible en verde.

Ahora en vez de **ver** el efecto del chip 555 en nuestro circuito, podemos escucharlo, utilizando un buzzer pieoelectrico pasivo!

1.5. Desafío Final:

- Utilizando el IC 555 y un potenciómetro, crea un emisor de sonido con frecuencia variable! Descubre cómo conectarlo para que la frecuencia cambie!
- Utilizando el IC 555, crea un mini teclado de cuatro notas, prueba distintas resistencias y capacitores hasta que te encuentres satisfecho con el resultado!

2. Bases de la Programación:

2.1. ¿Qué es un lenguaje de programación?

“Un lenguaje de programación es un lenguaje formal (o artificial, es decir, un lenguaje con reglas gramaticales bien definidas) que le proporciona a una persona, en este caso el programador, la capacidad de escribir (o programar) una serie de instrucciones o secuencias de órdenes en forma de algoritmos con el fin de controlar el comportamiento físico o lógico de un sistema informático, de manera que se puedan obtener diversas clases de datos o ejecutar determinadas tareas”

El lenguaje de programación de Arduino está basado en C, el cual está dentro de los más populares por su eficiencia y control. Sin embargo, es más complejo que otros como Python.

2.2. Estructura General

Primero hay que entender los 3 entornos básicos en los que se trabaja:

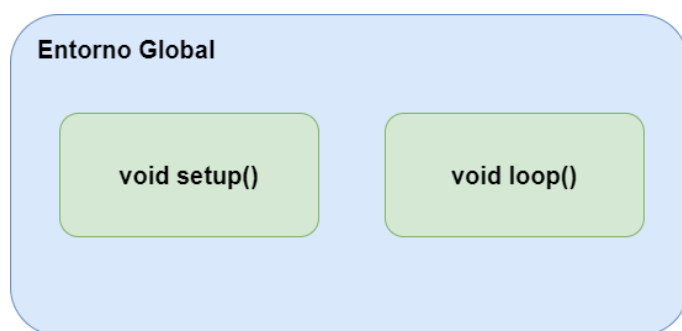


Figura 13: Diagrama del entorno de programación Arduino.

2.2.1. Entorno Global

Esta es la zona de código que se encuentra afuera de los voids. Aquí se suelen incorporar las librerías, definiciones y variables de tipo global(más adelante se profundiza). Básicamente aquí va todo lo que el código va a necesitar acceder desde diferentes entornos locales.

2.2.2. void setup()

El *void setup()* es lo que se conoce como un entorno local, y este es el primero que se ejecuta en el Arduino. Tal como dice su nombre (*setup = configuración*), es donde se realiza las configuraciones necesarias para trabajar y sólo se ejecuta 1 vez.

2.2.3. void loop()

Este es el segundo entorno local base de nuestro código y es donde lo principal se va a ejecutar. Tal como dice su nombre (*loop = bucle*) este entorno se ejecuta indefinidamente una y otra vez.

2.3. Variables

Las variables son la unidad base de información de nuestro código. En ellas se pueden guardar todo tipo de datos para poder manipularlas, tomar decisiones, etc.

2.3.1. Variables globales

Las variables globales son las creadas en el entorno global del código, estas variables pueden ser accedidas desde cualquier entorno local y su valor va a ser el mismo. Si un entorno local la modifica, el resto de entornos va a ver este cambio. Es importante que estas variables sean únicas, es decir, cuando se crean, no se puede crear otra con el mismo nombre dentro de ningún entorno.

2.3.2. Variables locales

Las variables locales son las que se crean dentro de los entornos locales. Estas variables sólo existen dentro del entorno donde se crearon y no pueden ser utilizadas en uno diferente. 2 entornos locales diferentes pueden tener una variable con el mismo nombre, la información que tenga una puede ser diferente de la otra y si se cambia, no va a afectar al resto.

2.3.3. Tipos de variables

Para crear las variables, se tiene que definir el tipo de información que va a ir dentro, si luego se intenta guardar algo de diferente tipo, se genera un error. En la siguiente tabla se ven los tipos de variables que más vamos a utilizar:

Tipo	Descripción	Ejemplo
int	Número entero	int numero = 7;
float	Número decimal	float decimal = 3.14;
char	Carácter	char letra = 'a';
String	Cadena de caracteres (palabras)	String palabra = "Hola mundo";
bool	Valor de verdad (verdadero o falso)	bool flag = true;

Tabla 2: Tipos de variables

¿Sabías qué? Existen muchos más tipos de variables, las cuales los puedes encontrar en la documentación de Arduino junto a muchas cosas más.

www.arduino.cc/reference/es/

2.3.4. Arreglos (listas)

Hasta ahora hemos visto que las variables nos permiten guardar información, sin embargo, sólo podemos guardar un elemento por variable. Para abordar esto, existe algo llamado arreglos, o también conocidos como listas, que permiten guardar varios elementos del mismo tipo en la misma variable.

Para crearlas, es necesario definir el tipo, el nombre, y el tamaño (entre corchetes []):

```
1 int lista_de_numeros[5];
2 int otra_lista_de_numeros[]={1,2,3};
3 float lista_de_decimales[10];
4 char lista_de_letras[] = {"h","o","l","a"};
```

Notar que se pueden crear arreglos vacíos o ya con datos dentro.

La forma en cómo están ordenados los elementos dentro de un arreglo es comenzando desde el 0, es decir, para una lista con 5 elementos, estos van a estar numerados del 0 al 4:

lista_de_numeros					
Índice	0	1	2	3	4
Valor	3	1	4	1	6

Ahora veamos un ejemplo básico de cómo se podrían utilizar los arreglos:

Primero creamos una lista de tamaño 2:

```
1 int lista[2];
```

Luego le asignamos valores a cada elemento:

```
1 lista[0] = 3;
2 lista[1] = 2;
```

Ahora con la lista llena de valores, se pueden realizar distintas operaciones, por ejemplo, una suma simple:

```
1 int resultado = lista[0] + lista[1];
```

En este caso, el valor guardado en resultado va a ser igual a 5 (2 + 3).

2.4. Lógica proposicional

La lógica proposicional hace referencia a aquellas afirmaciones que se pueden definir como verdadero o falso (“¿Esto es igual a esto otro?”, “¿Esto es más grande que esto otro?, etc”) y cómo interactúan unas con otras.

Dentro de Arduino tenemos 2 comparadores lógicos, el && y el ||.

2.4.1. Comparador lógico &&

Este comparador significa “y”. Se utiliza en preguntas del tipo: “Esta afirmación y esta otra afirmación”. Las combinaciones posibles se muestran a continuación:

Afirmación A	Afirmación B	A && B
Verdadero	Verdadero	Verdadero
Verdadero	Falso	Falso
Falso	Verdadero	Falso
Falso	Falso	Falso

Tabla 3: Tabla de verdad para el Y

2.4.2. Comparador lógico ||

Este comparador significa “o”. Se utiliza en preguntas del tipo: “Esta afirmación o esta otra afirmación”. Las combinaciones posibles se muestran a continuación:

Afirmación A	Afirmación B	A B
Verdadero	Verdadero	Verdadero
Verdadero	Falso	Verdadero
Falso	Verdadero	Verdadero
Falso	Falso	Falso

Tabla 4: Tabla de verdad para el O

2.5. Aritmética

2.5.1. Comparación Aritmética

Cuando estamos trabajando con variables de tipo numéricas, podemos realizar comparaciones entre 2 variables y así poder crear afirmaciones. Las diferentes formas de comparación se pueden ver a continuación:

Nombre	Símbolo
Igual	A == B
Desigual	A != B
Menor estricto	A < B
Menor o Igual	A <= B
Mayor estricto	A > B
Mayor o Igual	A >= B

Tabla 5: Comparadores aritméticos

2.5.2. Operadores aritméticos

Por último, como estamos trabajando con números, podemos realizar operaciones matemáticas con ellos como sumar, restar, multiplicar y dividir(+, -, *, /).

```

1  int a = 1;
2  int b = 5;
3  int c;
4  c = a + b;

```

En este ejemplo, el valor de la variable *c* va a ser 6. Para evitar algún tipo de problema, evita sumar variables de distinto tipo (Por ejemplo: *int* + *float*).

2.6. Consola Serial

Para poder visualizar mejor los contenidos siguientes, vamos a ver una pequeña introducción a la consola serial, más adelante en la Unidad 4 lo profundizaremos.

Con el siguiente código y mediante el *Serial.println()* vamos a poder mostrar cosas por la consola serial de Arduino:

```

1  int variable = 1;
2
3  void setup(){
4      Serial.begin(9600);
5  }
6
7  void loop(){
8      Serial.println("Una frase");
9      Serial.println(variable);
10 }

```

2.7. Estructuras condicionales

Las estructuras condicionales son las herramientas que tenemos para realizar diferentes acciones y controlar el flujo de nuestro código.

Estas funcionan en base a nuestras variables y básicamente son preguntas que podemos realizar dentro del código.

2.7.1. if

La estructura más básica de todas es el *if* que significa “Si esto es verdad”.

La estructura consta de un *if*, una condición entre paréntesis, y lo que se quiere hacer en caso de que la condición sea verdadera entre llaves:

```

1  if(condición){
2      //hacer algo
3  }

```

La condición debe ser algo que pueda ser verdadero o falso, la más típica a utilizar es la de verificar si una variable es igual a algo, lo cual se logra con el símbolo “==”:

```

1  numero == 3 //se pregunta si el número es
    igual 3

```

También se pueden encadenar muchos *if* con la estructura *if-else if* de la siguiente manera:

```

1  int numero = 2;
2  if(numero == 1){
3      //este pedazo de código se va a ignorar
        porque la condición es falsa
4  }else if(numero == 2){
5      //este código si se va a ejecutar porque
        la condición es verdadera
6  }else if(numero == 3){
7      //esta condición no se alcanza a verificar
        porque ya se encontró la verdadera
8  }

```

Por último, esto se puede complementar con la estructura *if-else if-else*, donde el último *else* está para considerar todo el resto de casos:

```

1  int numero = 2;
2  if(numero == 1){
3      //este pedazo de código se va a ignorar
        porque la condición es falsa
4  }else if(numero == 3){
5      //este pedazo de código se va a ignorar
        porque la condición es falsa
6  }else{
7      //este código es el que SI se ejecuta
8  }

```

Ahora en TinkerCad, prueba el siguiente código y varía las variables *a* y *b*, para modificar la salida (también prueba añadiendo un *if-else*):

```

1  int a = 10;
2  int b = 5;
3
4  void setup(){
5      Serial.begin(9600);
6  }
7
8  void loop(){
9      if (a<b){
10         Serial.println("A es mayor que B");
11     }else{
12         Serial.println("El if es FALSO");
13     }
14     while(true);
15 }

```

WARNING:

- El *if* puede ir solo.
- Se pueden tener tantos *else if* como se quieran, pero siempre deben ir acompañados de un *if* al principio
- Sólo se puede tener un único *else*, y puede ir inmediatamente después de un *if* o un *else if*

2.7.2. while

El *while* es muy parecido al *if*, sólo que significa “Mientras la condición sea verdadera”, o sea, que va a ejecutar indefinidamente hasta que la condición se vuelva falsa.

Es importante fijarse que al momento de establecer la condición, esta en algún momento se vuelva falsa, ya que si no, el código se va a quedar atrapado por siempre dentro del **while**.

```
1  int contador = 0;
2  while(contador<5){
3      //en este caso el código se quedará
        ejecutando eternamente ya que contador
        no cambia.
4  }
```

```
1  int contador = 0;
2  while(contador<5){
3      contador = contador + 1;
4      //en este caso, el código se ejecutará 5
        veces y luego saldrá del while
5  }
```

Ahora para practicar, prueba el siguiente código en TinkerCad y juega con él:

```
1  int a = 0;
2
3  void setup(){
4      Serial.begin(9600);
5  }
6
7  void loop(){
8      while (a<10){
9          Serial.println("Estoy en el While");
10         Serial.println(a);
11         a += 2;
12     }
13     while(true);
14 }
```

2.7.3. for

La última estructura que vamos a ver es bastante compleja, pero cuando se utiliza bien es bastante poderosa, el *for*.

A diferencia del **if** o el **while**, dentro de los paréntesis no va únicamente una condición, si no que lo siguiente: variable, condición, modificador:

```
1  for(variable;condición;modificador){
2      //Código del for
3  }
```

En el código anterior, primero se declara una variable, la cual usualmente es llamada *i*, inicializada en 0:

```
1  for(int i = 0;condición;modificador){
2      //Código del for
3  }
```

Luego se declara la condición, la cual debe contener a la variable anteriormente creada (*i*):

```
1  for(int i = 0; i < 3;modificador){
2      //Código del for
3  }
```

Y por último el modificador, el cual realiza una operación sobre la variable (*i*) en cada iteración del *for*. Este modificador puede ser cualquier operación matemática que se quiera, con la precaución de que en algún momento la condición se hará falsa (como en el **while**):

```
1  for(int i = 0; i < 3;i++){
2      //Lo que está dentro del for se ejecutará
        3 veces
3  }
```

En este caso se utiliza *i++*, lo cual significa que en cada iteración se sumará 1 a la variable, también existe *i--*, que resta, o se puede usar cualquier otra operación matemática de la forma *i = <operación>*.

Ahora para practicar el *for* y reforzar el concepto de listas, ejecuta el siguiente código en TinkerCad. Prueba jugar con listas de diferente tipo.

```
1  int lista[] = {3,1,4,1,5};
2  int largo_lista = 5;
3
4  void setup(){
5      Serial.begin(9600);
6  }
7
8  void loop(){
9      for(int i = 0; i < largo_lista;i++){
10         Serial.println("Mostrar item lista");
11         Serial.println(lista[i]);
12     }
13     while(true);
14 }
```

2.8. Funciones

Como vimos antes, existen 2 entornos locales nativos en Arduino, pero nosotros podemos crear nuestros propios entornos para hacer nuestro código más ordenado.

Estos entornos propios se denominan *Funciones*, las cuales deben ser definidas dentro del entorno global, es decir, no pueden ser creadas dentro del **void setup()** o el **void loop()**.

De esta forma podemos escribir códigos complejos de tareas que se hagan repetidamente, y así sólo usamos la función en vez de escribir mucho código cada vez que necesitemos hacer algo.

2.8.1. Funciones básicas

La función más básica que se puede crear es de tipo void, lo que significa que no nos va a devolver ningún valor, sólo realiza una acción:

```
1 void mi_funcion(){
2     //aquí va el código de la acción que se
3     quiere hacer
}
```

Y luego cuando se quiera utilizar esta función, se debe llamar en el **void loop()** o en el **void setup()** de la siguiente manera:

```
1 void loop(){
2     mi_funcion();
3 }
```

Desde esta función se podrá acceder sólo a las variables globales y las que se creen dentro de la misma, pero no a las creadas en el **void loop()** o en el **void setup()**.

2.8.2. Funciones con entrada(Input)

Cuando necesitemos pasar información a nuestra función desde el entorno del que se llama, se pueden establecer lo que se conoce como inputs.

Con esto le decimos a la función de que tiene que recibir un dato, el cuál tiene que ser de un tipo específico que le indiquemos dentro de los paréntesis junto a un nombre para utilizarlo dentro de la función:

```
1 void mi_funcion_input(int a, char b){
2     //aquí va el código de la acción que se
3     quiere hacer
4     //se pueden utilizar las variables a y b
}
```

Notar que si se quieren usar más de un input (pueden ser todos los que quieran), se deben separar con una coma. Luego cuando se quiera utilizar esta función, se debe llamar en el **void loop()** o en el **void setup()** de la siguiente manera, preocupándose de pasarle los datos necesarios:

```
1 void loop(){
2     mi_funcion_input(1,"b");
3 }
```

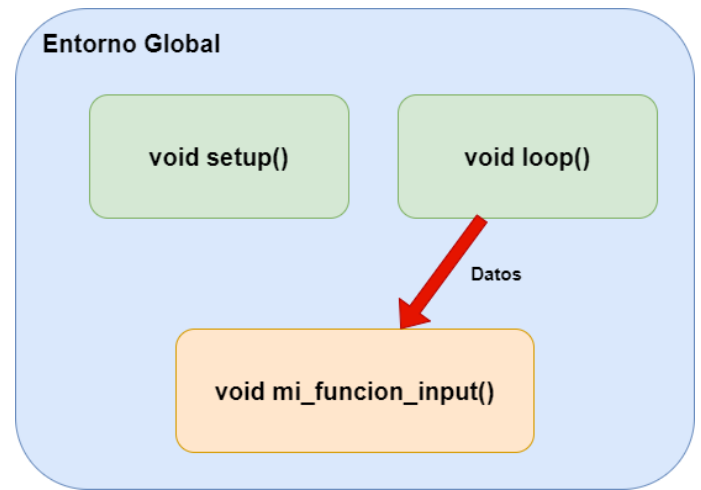


Figura 14: Diagrama de una función con input.

2.8.3. Funciones con salida(Output)

Ahora tenemos el caso contrario, en el que queramos obtener la información generada dentro de una función, esto se denomina como el retorno de la función, y para lograr esto, se tiene que especificar el tipo de dato que se quiere devolver, quedando algo de la siguiente manera:

```
1 int mi_funcion_output(){
2     int variable = 1;
3     //aquí va el código de la acción que se
4     quiere hacer
5     return variable;
}
```

Notar que en vez de void, se escribe el tipo de variable que se quiere retornar. Consideren que no se pueden retornar más de una variable de esta forma. También es importante fijarse en cómo se utiliza dentro del **void loop()** o en el **void setup()**:

```
1 void loop(){
2     int variable;
3     variable = mi_funcion_output();
4 }
```

Fíjense que la función debe ser asignada a una variable del mismo tipo que retorna para guardar la información.

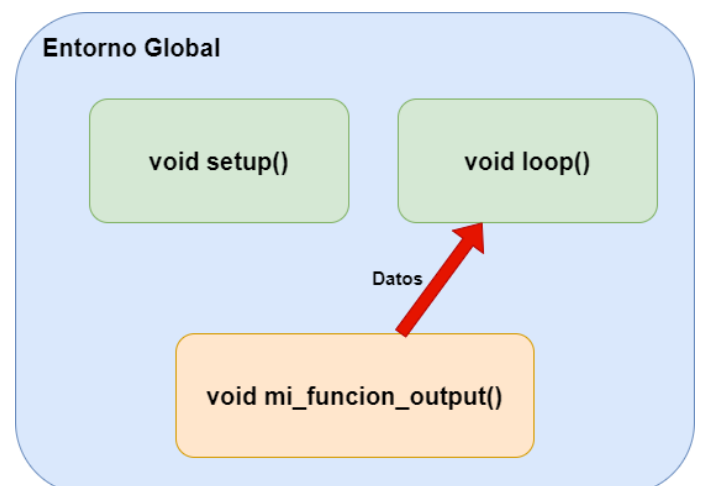


Figura 15: Diagrama de una función con output.

2.8.4. Funciones Input-Output

Por último, se pueden crear funciones que reciban información, las procesen, y retornen un resultado, lo cual se logra uniendo los dos métodos vistos anteriormente, por ejemplo, en esta función de suma:

```
1  int suma(int a,int b){
2      int resultado = a + b;
3      return resultado;
4  }
```

2.9. Desafíos

Determina el valor de verdad de las siguientes expresiones:

1. (Verdadero && Falso) || Verdadero
2. (false || false) || true
3. true && (false && false)

Determina el valor del elemento faltante para que toda la expresión sea verdadera:

1. (Verdadero && ???) && Verdadero
2. ??? || ((true && false) || false)

Determina el valor del elemento faltante para que toda la expresión sea falsa:

1. (??? || Falso) && Verdadero
2. ((true && false) || true) && ((true && ???) && true)

Determina qué parte del código se va a ejecutar:

```
1  float a = 3.14;
2  float b = 3.09;
3  bool c = false;
4  if(a < b){
5      //parte 1
6  }else if((a > b) && c){
7      //parte 2
8  }else{
9      //parte 3
10 }
```

```
1  void setup(){
2      float a = 2.5;
3      int z = funcion(a);
4      if (z==1){
5          //parte 1
6      }else if(z!=2){
7          //parte 2
8      }else{
9          //parte 3
10     }
11 }
12
13 int funcion(float x){
14     if(x < 1){
15         return 1
16     }else if (x > 2.49){
17         return 2
18     }else{
19         return 3
20     }
21 }
```

Determina cuántas veces se va a ejecutar cada código:

```
1  int i = 3;
2  while(i < 6){
3      //codigo
4      i = i - 1;
5  }
```

```
1  int i = 3;
2  while(i < 6){
3      for(int j=0; j<3;j++){
4          //codigo
5      }
6      i = i + 1;
7  }
```

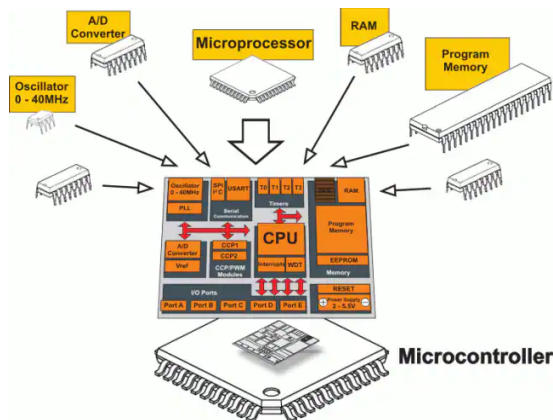
Determina cuántas veces se va a ejecutar cada parte del código y en qué orden:

```
1  int f = 5;
2  for(int i = 0; i<6;i++){
3      if(i == f){
4          //parte 1
5      }else if(i<3){
6          //parte 2
7      }else{
8          //parte 3
9      }
10 }
```

3. Circuitos Programables:

3.1. Microcontroladores:

Un microcontrolador es esencialmente un pequeño computador, capaz de recibir información (*input*), procesarla y generar una respuesta (*output*) en base a instrucciones previamente determinadas. Todo microcontrolador posee una serie de componentes básicos que le permiten cumplir diversas funciones:



- **CPU:** La *Unidad de procesamiento central*, es el cerebro del microcontrolador, el componente mediante el cual monitorea la información que recibe y decide las acciones que debe llevar a cabo.
- **RAM:** La *Memoria de acceso aleatorio*, es el espacio donde se almacena la información recibida para que el procesador tenga acceso rápido a esta, para realizar cálculos y tomar decisiones.
- **ROM:** La *Memoria de sólo lectura* es un componente donde se almacena la información que el procesador utiliza para llevar a cabo sus cálculos y acciones, no corresponde a las instrucciones generadas por un usuario, son las instrucciones básicas necesarias para que el microcontrolador funcione.
- **I/O:** Los *Puertos de entrada y salida* son uno o más conectores donde el microcontrolador recibe información en forma de señales eléctricas, para que la CPU las procese.
- **Oscilador:** Componente que define la velocidad y la frecuencia bajo la cual la CPU procesará la información y tanteará las señales recibidas en los puertos de entrada y salida.
- **EEPROM** La *ROM programable y borrable eléctricamente* es el componente donde se almacenan las instrucciones programadas en el microcontrolador, para alterar su funcionamiento y definir las acciones a llevar a cabo dependiendo el input recibido, con el objetivo de solucionar un problema específico.

Existen muchos tipos de microcontroladores, donde variará la complejidad de cada uno de estos componentes, dependiendo de las aplicaciones para las que hayan sido diseñados.

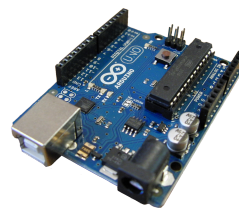
3.2. Arduino:

Arduino es una compañía que ofrece productos y servicios relacionados al software y hardware de código abierto. esencialmente, Arduino ofrece dos productos que se complementan entre sí:

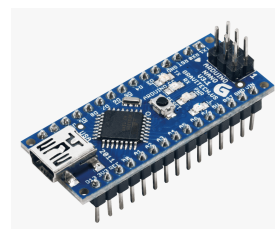
- **Hardware:** Placas de desarrollo "*Plug and Play*". Estas consisten de microcontroladores en una placa de circuitos, con todos los componentes necesarios para ser utilizadas de manera inmediata con un computador, incluyendo interfaces para poder programarlas de manera directa, reguladores de energía, y acceso fácil a las entradas y salidas mediante conectores estandarizados.

Arduino ofrece distintos modelos dependiendo del microprocesador utilizado, y el uso al que iría orientado el circuito (hobby, profesional, inteligencia artificial, ropa, etc...). A continuación, veremos algunos de los modelos más conocidos, utilizados y disponibles en el mercado:

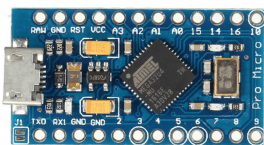
Arduino UNO: Basado en el microcontrolador *ATmega328p* como unidad central y el *ATmega16U2* como interfaz USB, es la primera placa desarrollada por Arduino y revolucionó la accesibilidad a hobbyistas sin conocimientos necesarios como para fabricar su propia placa, facilitando el enfoque de su desarrollo en la implementación y no en la electrónica. Ya lo conocemos, como el cerebro del IROH, utilizados en los talleres básico e intermedio.



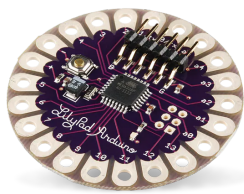
Arduino nano: Basado en el microcontrolador *ATmega328p* o *ATmega168p* como unidad central (32 vs 16 kB de memoria) y el *AFT232RL* como traductor de la comunicación serial a USB, fue diseñado como una versión compacta del Arduino UNO, útil para desarrollo de prototipos utilizando protoboards o placas personalizadas más complejas. Hemos utilizado este componente tanto en los clubes de robot velocista como en arte tecnológico.



Arduino micro y pro-micro Basado en el microcontrolador **ATmega32u4**, que posee comunicación USB integrada y que facilita el desarrollo de dispositivos de interfaz humana, como teclados, mouse y controladores MIDI. Un clon desarrollado por *SparkFun* (pro-micro) es ampliamente utilizado por el hobby de desarrollo de teclados mecánicos.



Arduino lillypad: Basado en el microcontrolador **ATmega328v** o **ATmega168v**, con el propósito de obtener una placa que facilite la creación de ropa tecnológica. Esta placa está diseñada para ser fácilmente escondida o bordada en ropa que posea capacidades tecnológicas como bio-sensores, luces y otros dispositivos.



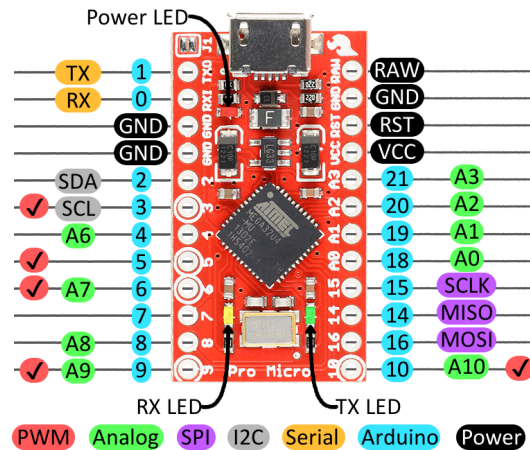
Aparte de estos, existen muchos más, que puedes encontrar en www.arduino.cc/en/hardware.

- **Software:** Entornos de desarrollo, donde se puede escribir programas, administrar librerías y subir instrucciones al hardware compatible. Arduino desarrolló su propio lenguaje de programación basado en **C++** y **Processing**, que facilita el desarrollo de programas enfocados al uso de microcontroladores. Existen dos versiones del entorno de desarrollo, Arduino 1.X y Arduino 2.X. Durante el desarrollo de este club, utilizaremos la versión 2.X, la más actualizada de todas.

Puedes encontrar todas las versiones de software disponibles en www.arduino.cc/en/software.

3.3. Arduino Pro Micro:

Durante la clase pasada ya conocimos a la placa de desarrollo que utilizaremos durante este semestre, el **Arduino Pro Micro**, a continuación veremos las capacidades que tiene:



Esta figura corresponde al **pinout** de la placa de desarrollo, donde se especifican las capacidades de cada una de las entradas y salidas del dispositivo, a continuación, se presenta una breve explicación de cada etiqueta y su color.

- **PWM:** Pines capaces de emitir señales de tipo PWM o "modulación por ancho de pulsos", que permite emular una emisión de señales análogas. Los utilizaremos para controlar servomotores, buzzer piezoeléctricas y cualquier dispositivo que tenga un control análogo.
- **Analog:** Pines con la capacidad de leer señales análogas y transformarlas a un valor digital, los utilizaremos para utilizar sensores que entreguen valores distintos a encendido y apagado (sensores análogos).
- **SPI:** Pines con la capacidad de comunicarse con otros dispositivos mediante el protocolo SPI (no lo veremos durante el taller).
- **I2C:** Pines con la capacidad de comunicarse con otros dispositivos mediante el protocolo I2C, los utilizaremos para controlar una pequeña pantalla OLED.
- **Serial:** Pines que permiten comunicarse con otros dispositivos mediante el protocolo serial, en general utilizamos este protocolo para comunicarnos con el computador mediante USB, pero también puede utilizarse para controlar, y transmitir información entre dos dispositivos.
- **Arduino:** Corresponden a los pines capaces de recibir y emitir señales digitales, serán los pines más utilizados durante el club.
- **Power:** Corresponden a los pines relacionados con la energía eléctrica, que permiten alimentar el arduino desde fuentes externas, distribuir energía a otros dispositivos y reiniciar el procesador de manera eléctrica.

Veamos más en detalle que es lo que significan y como utilizar estos pines.

3.4. Energía:

El arduino pro micro tiene cuatro tipos de pines asociados a energía (etiquetados como **Power** en el pinout), sus funciones y la manera de usarlo son las siguientes:

- **RAW:** Pin mediante el cual se puede alimentar al arduino directamente con un voltaje de entre 5 y 12 [V], en caso de no utilizar el puerto USB para proveerlo de energía.
- **VCC:** Corresponde al terminal positivo de la fuente de energía que la placa de desarrollo es capaz de proveer, sirve para alimentar componentes que necesitan energía de manera constante.
- **GND:** Corresponde al terminal negativo de la fuente de energía o tierra del circuito, todos los componentes y circuitos deben ir conectados a este pin por su terminal negativo.
- **RST:** Pin de reinicio del microcontrolador, cuando este pin es conectado a tierra (**GND**), el procesador se reiniciará.

En resumen, los principales terminales que debemos conocer son **VCC** y **GND**, que actuarán como terminales de una batería para alimentar a ciertos componentes a medida que los utilicemos.

3.5. Señales Digitales:

Las señales digitales son aquellas señales que solo tienen dos estados **encendido** o **apagado**, estos dos estados pueden tener varios nombres:

Encendido	Apagado
HIGH	LOW
ON	OFF
True	False
1	0

El arduino pro micro es capaz de emitir y leer señales digitales a través de sus 18 pines digitales, etiquetados como **Arduino** en el pinout. Veamos que es lo que significan estas dos capacidades, y como realizarlas utilizando funciones predeterminadas:

3.5.1. Lectura de señales digitales:

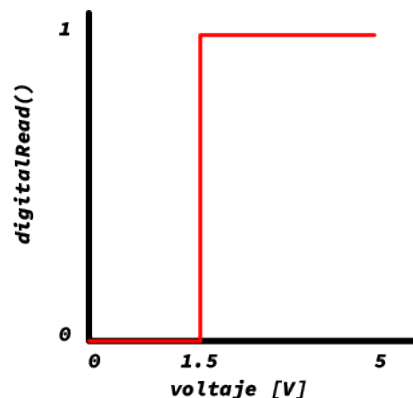
Para configurar unos de los pines digitales como entrada, es necesario utilizar la función de configuración, con el parámetro **INPUT**.

```
1 void setup(){
2   pinMode(pin, INPUT);
3 }
```

Una vez establecido el modo de operación de un pin como entrada digital, podemos obtener el estado que está leyendo utilizando la función :

```
1 int estadoPin = 0;
2
3 void loop(){
4   estadoPin = digitalRead(pin);
5 }
```

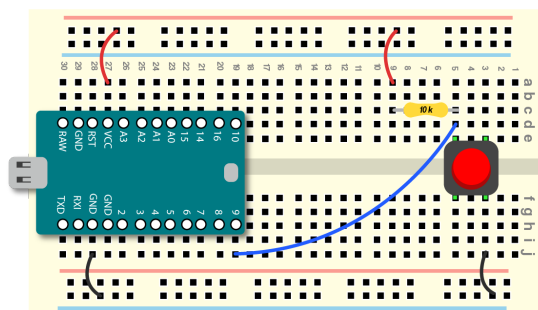
El valor de voltaje que determina cuando la señal está encendida, depende del microprocesador utilizado y su voltaje de operación, en el caso del utilizado por el arduino pro micro es aproximadamente 1.5 [V].



Ejercicio: Vamos a conectar un interruptor directamente al pin, para controlar diversos dispositivos. Para leer la señal proveniente desde este interruptor, podemos conectarlo de dos maneras:

- **PULLUP:** Conectamos VCC al pin que utilizaremos mediante una resistencia de 10K. Conectamos VCC a GND mediante el interruptor.

De esta manera, el voltaje proveniente de la fuente de poder siempre está conectada al pin mediante la resistencia y cuando apretamos el interruptor, el voltaje bajará en el pin, ya que el camino hacia el terminal negativo del arduino (GND) ofrece menos resistencia.

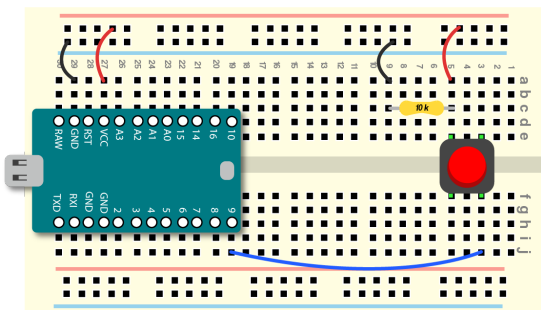


Cuando conectamos el interruptor de esta manera, obtendremos los siguientes resultados de **digitalRead()**:

Interruptor	<i>digitalRead()</i>
OFF	1
ON	0

- **PULLDOWN:** Conectamos VCC a GND mediante una resistencia de 10K. Conectamos VCC al pin que utilizaremos mediante el interruptor.

De esta manera, el voltaje proveniente de la fuente de poder siempre está conectada al terminal negativo (GND) mediante la resistencia y cuando apretamos el interruptor, el pin observará un voltaje positivo, ya que el camino hacia este ofrece una menor resistencia.



Cuando conectamos el interruptor de esta manera, obtendremos los siguientes resultados de `digitalRead()`:

Interruptor	<code>digitalRead()</code>
OFF	0
ON	1

Ahora prueba estos dos circuitos, y comprueba el comportamiento que acabamos de mencionar, utilizando el **monitor serial**.

3.5.2. Emisión de señales digitales:

Para configurar unos de los pines digitales como salida, es necesario utilizar la función de configuración, con el parámetro `OUTPUT`.

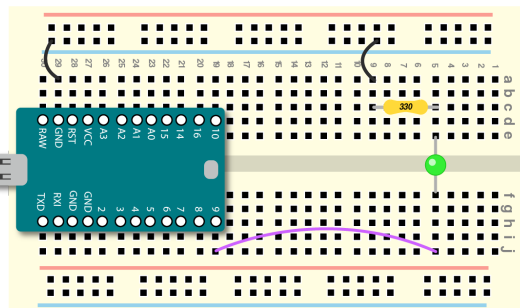
```
1 void setup(){
2   pinMode(pin, OUTPUT);
3 }
```

Una vez configurado, el pin podrá ser utilizado como una fuente emisora de señal digital, es decir, se podrá controlar de manera precisa cuando se enciende y se apaga el voltaje desde ese pin, con esto podremos controlar el encendido y apagado de cualquier componente digital, que requiera una señal de 5v y podremos comunicarnos con dispositivos que tengan algún tipo de comunicación por señales digitales (como múltiples tipos de sensores y pantallas).

Utilizando la función `digitalWrite()`, podrás encender o apagar dispositivo que requieran de un voltaje constante.

```
1 void loop(){
2   // Encender el pin digital
3   digitalWrite(pin, HIGH);
4   // Apagar el pin digital
5   digitalWrite(pin, LOW);
6 }
```

Ejercicio: Ejercitemos el uso de esta función implementando el ejercicio más simple **BLINK**, conectemos un LED y una resistencia como se ve en el diagrama, y completen los siguientes desafíos:



- **1) Blink Básico:** Hacer parpadear la luz con un intervalo de 1 segundo.
- **2) SOS:** Hacer parpadear el LED con un patrón de SOS: tres parpadeos cortos, una pausa corta, tres parpadeos largos, una pausa corta, tres parpadeos cortos y una pausa larga (.../—/...).
- **3) Acelerando:** Hacer parpadear la luz cada vez más rápido, utilizando un *ciclo for* o un *ciclo while*, partiendo con la luz encendida por 2 segundos acortando el tiempo cada vez que se apaga.

Repitamos este ejercicio, pero utilizando el buzzer activo.



Ejercicio: Ahora, mezclemos las dos capacidades (input y output digital) para controlar distintos LEDs. Para facilitar el desarrollo de este ejercicio y disminuir la cantidad de componentes utilizados, revisemos la siguiente información.

Existe también el modo **INPUT_PULLUP** en arduino, que activa un circuito de tipo pullup, conectado directamente al pin digital, ahorrando la necesidad de armar el circuito de manera externa.

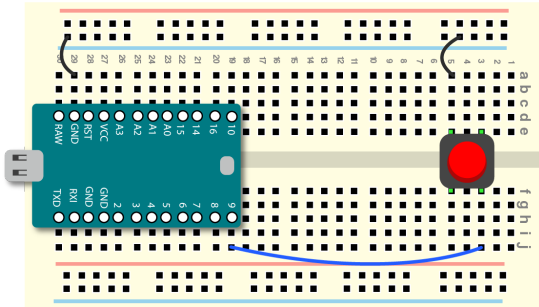
Simplemente conecta un terminal del interruptor al pin de la placa, y otro terminal a tierra, y podrás utilizarlo tal y como si hubieras hecho el circuito que probamos en el ejercicio anterior. Procura utilizar la siguiente linea en el `setup()`:


```

1 void setup(){
2   pinMode(pin, INPUT_PULLUP);
3 }

```

y conecta de la siguiente manera:



Sabiendo esto, intenta resolver los siguientes problemas:

1. **ON/OFF:** Utiliza botón para encender y apagar una luz LED, de la siguientes dos maneras:
 - La luz LED deberá encenderse solo cuando el botón está apretado
 - La luz LED deberá encenderse cuando se apriete el botón y deberá apagarse cuando se vuelva a apretar el botón.
2. **Semaforo:** Conecta tres LEDs a tres pines distintos, e intenta simular un semáforo, haciendo que el LED que esté encendido, cambie cada vez que se aprieta el botón.
3. **Contador:** Con los mismos tres LEDs, intenta implementar un contador, que vaya aumentando la cantidad de luces encendidas cada vez que aprietas el botón, una vez que llegues a 4, apáguelas todas.
4. **Contador Binario:** Con los mismos tres LEDs, implementa un contador binario, que encienda y apague las luces representando la cantidad de veces que se haya apretado el botón con un número binario, como se ve en los siguientes ejemplos:

Veces Apretado	LED 1	LED 2	LED 3
0 = 000	OFF	OFF	OFF
1 = 001	OFF	OFF	ON
2 = 010	OFF	ON	OFF
3 = 011	OFF	ON	ON
4 = 100	ON	OFF	OFF
5 = 101	ON	OFF	ON
6 = 110	ON	ON	OFF
7 = 111	ON	ON	ON

3.6. Señales Análogas:

Las señales análogas, son aquellas señales que pueden tomar cualquier valor posible, y no están limitadas a encendido y apagado, en el caso de la electricidad, correspondería al valor del voltaje de la señal. Las placas de desarrollo arduino poseen un componente llamado “**Convertor Análogo a Digital (ADC)**”, que permite leer valores análogos desde componentes electrónicos, y almacenar los como un valor numérico. La precisión de estos ADC dependen de la cantidad de bits en la que pueden dividir la señal.

3.6.1. Lectura de señales análogas:

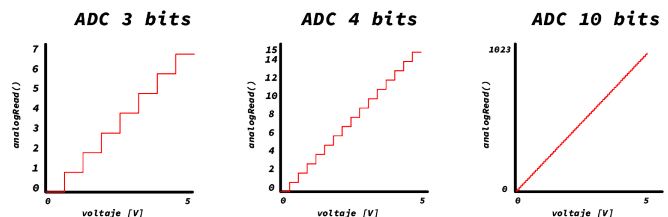
Para leer señales análogas desde un pin, no es necesario configurarlo en el `setup()`, sólo se requiere que el pin sea de tipo análogo, es decir, que esté etiquetado como [AX]. La lectura se realiza utilizando la siguiente función:

```

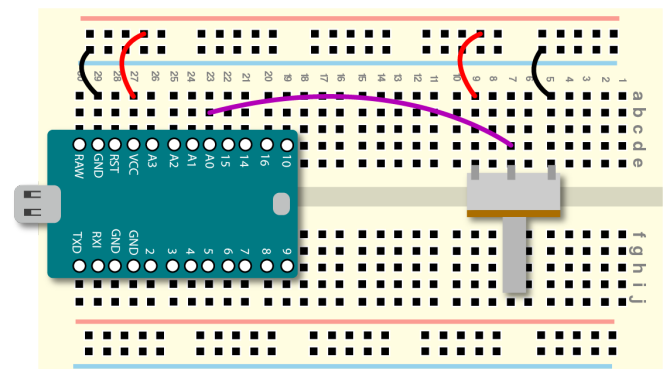
1 int estadoPin;
2
3 void loop(){
4   estadoPin = analogRead(AX);
5 }

```

Esto entregará un valor entre 0 y 1023, ya que nuestro arduino posee un ADC (Convertor analógico a digital) de 10 bits. Este convertor funciona midiendo el voltaje recibido y transformándolo a un número que podemos interpretar en nuestro código. Mientras más bits tiene, se tendrá una mayor resolución a la hora de medir el voltaje que recibe el pin, tal y como se muestra en la siguiente figura:



Ejercicio: Observemos como funciona la lectura de señales análogas, conectando una resistencia variable (potenciómetro) y viendo los valores que aparecen en la pantalla. Sigamos el siguiente diagrama:



Observemos los valores que arduino lee utilizando el **monitor serial**.

Ejercicio: Utilicemos el potenciómetro para controlar el comportamiento de uno o varios LEDs

1. **Blink 2:** Utiliza los valores del potenciómetro para controlar la velocidad con la parpadea un LED.
2. **Indicador:** Utiliza el potenciómetro para controlar la cantidad de LEDs que se encienden a medida que lo giras, parte con 2 y ve si puedes llegar a 4. *Este desafío necesita que investigues el uso de la función `map()`, queda como parte del ejercicio buscar cómo funciona en internet y descubrir por qué sirve para solucionar esto! discútelo con tus compañeros y pide ayuda al mentor.*

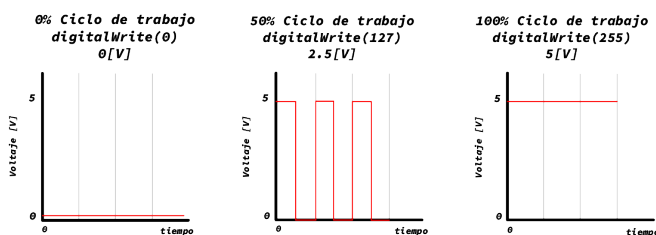
3.6.2. Escritura de señales analógicas:

La escritura de señales analógicas se realiza utilizando una técnica llamada **PWM** (*Pulse Width Modulation*, o *modulación por ancho de pulsos*).

Para emitir una señal analógica, no es necesario configurar el modo del pin en el setup, solo debe asegurarse que el componente al que se le enviará el voltaje esté conectado a alguno de los pines que pueden emitir PWM.

```
1 void loop(){
2   // valor = 0 -> 255
3   analogWrite(pinPWM, valor);
4 }
```

Los valores que arduino puede emitir van de 0 a 5 V, en 256 intervalos, es decir, puedes ir aumentando el voltaje en incrementos de 0.02 [V] aproximadamente.



Ejercicio: Utilicemos la función `analogWrite()` para controlar el brillo de un LED, a partir de la posición de la perilla de un potenciómetro. Utiliza los diagramas de los ejercicios anteriores para hacerte una idea de cómo conectar los componentes.

3.6.3. Otros usos de PWM

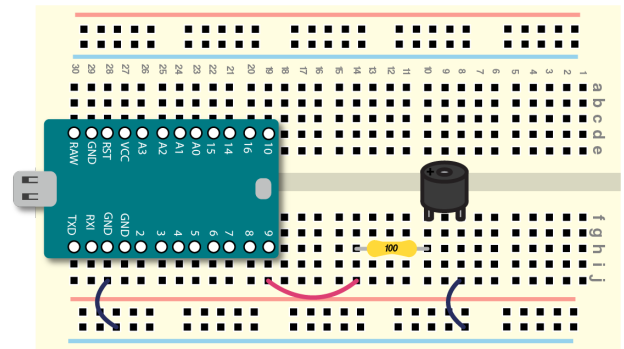
Existen distintas funciones y librerías que utilizan la capacidad de Arduino de modular pulsos, para controlar distintos tipos de dispositivos que pueden ser controlados o activados al variar el voltaje que se les entrega, podemos ver dos ejemplos simples: **buzzer piezoeléctrico pasivo** y **servomotores**.

3.6.4. buzzer piezoeléctrico pasivo

Podemos utilizar la función `tone()`, para generar un tono con alguna frecuencia, idealmente entre 200 y 5000 [Hz], para generar sonidos utilizando un buzzer piezoeléctrico pasivo. La manera en la que se utiliza esta función es la siguiente:

```
1 // Definir el pin
2 const int piezo = 9;
3
4 // Configurar el pin como output
5 void setup(){
6   pinMode(piezo, OUTPUT);
7 }
8
9 void loop(){
10  // Enviar un tono
11  tone(piezo, frecuencia);
12  // Apagar el tono
13  noTone(piezo);
14 }
```

El buzzer debe estar conectado de la siguiente manera:



Ejercicio: Tenemos los siguientes desafíos que involucran al buzzer piezoeléctrico.

1. **Canción:** Escribe una breve canción utilizando tonos de distintas frecuencias y delays, puedes hacerlo al oído o buscando las notas en internet. Algo útil es tener una lista de frecuencias y sus notas correspondientes, como por ejemplo la siguiente calculadora <https://www.translatorscafe.com/unit-converter/es-ES/calculator/note-frequency/>
2. **Instrumento:** Controla la nota emitida por el buzzer piezoeléctrico utilizando un potenciómetro.
3. **Instrumento 2:** Agrega un botón para que el piezo solo suene cuando se aprieta el interruptor.
4. **Instrumento 3:** transforma el sonido para que emita un arpeggio (investigar que es un arpeggio) en vez de una nota. Ahora estaríamos acercándonos a tener un sintetizador.

4. Análisis y diseño de algoritmos:

4.1. ¿Qué son los algoritmos?

“Un algoritmo es una secuencia de pasos lógicos que permiten solucionar un problema, dado un estado inicial y una entrada, para llegar a un estado final y una salida”

Como hemos visto, la robótica es la unión de la electrónica y la programación, y a medida que se nos vayan presentando diferentes problemas, las soluciones se vuelven más complejas, es por eso que es importante crear algoritmos útiles.

Para esto es crucial entender que se tiene una entrada, que es la información que se le da al algoritmo, un proceso, que es lo que realiza la tarea que se quiere con la información dada, y una salida, que es el resultado de nuestro algoritmo.



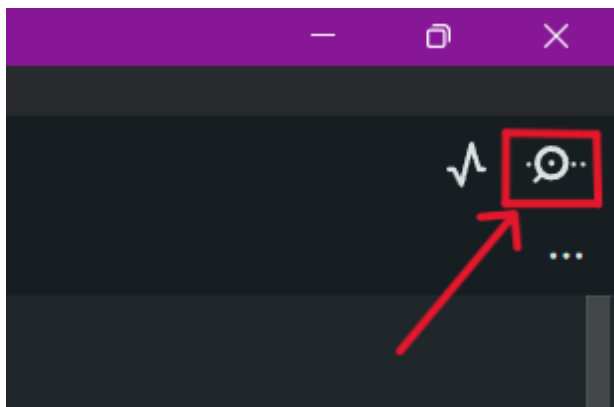
Esto se puede aplicar a muchos contextos más allá de la robótica, y en esta unidad nos vamos a enfocar en programar soluciones a problemas que no necesariamente son de robótica.

4.2. Comunicación Serial

Arduino nos ofrece una forma de comunicarnos con él a través de la pantalla del computador, la cual se conoce como **comunicación serial**.

El correcto uso de esta herramienta nos permitirá tener un mayor control sobre nuestros algoritmos y nos ayudará a detectar errores con mayor facilidad.

Para poder ver los mensajes de nuestro Arduino existe una herramienta dentro del programa llamada consola serial, ubicada en la esquina superior derecha, que básicamente es un chat con nuestro Arduino y sólo es necesario tenerlo conectado mediante USB:



4.2.1. Serial.begin()

El comando `Serial.begin()` es el que se encarga de inicializar la comunicación entre el Arduino y el computador. Debe ir en el `void.setup()`, y dentro de los paréntesis se especifica la velocidad de la conexión (BaudRate), la cual usualmente se establece en 9600.

```
1 void setup() {  
2   //Se inicializa la comunicación serial  
3   Serial.begin(9600);  
4 }
```

¿Sabías qué? *BaudRate* significa tasa de Baudios, y básicamente es el número de señales que se envían por segundo. Va a depender de cada modelo de placa qué velocidades soportan

4.2.2. Serial.print() y Serial.println()

Ya con la conexión establecida, podemos leer información desde el Arduino a través de la consola serial, para esto tenemos los comandos `Serial.print()` y `Serial.println()`, siendo la única diferencia que el último genera un salto de línea. Dentro de los paréntesis se puede poner el texto que queramos entre comillas dobles o simples, o directamente el nombre de una variable. A continuación se puede ver un ejemplo de cómo utilizar estos comandos:

```
1 int edad=15;  
2 Serial.print("Hola");  
3 Serial.println('tengo');  
4 Serial.print(edad);  
5 Serial.println(" años.");
```

Siendo su resultado:

```
Holatengo  
15 años.
```

4.2.3. Serial.available()

Ahora que sabemos cómo leer información desde el Arduino, hay que entender cómo se mandan datos hacia la placa. Esta tiene algo llamado *buffer* que básicamente es una cola donde esperan los caracteres que enviamos a través de la consola serial.

Es por esto que existe un comando para saber si hay datos en la cola, `Serial.available()` que básicamente se puede utilizar como la condición de un `if`, el cual retorna el número de caracteres que hay en la cola.

```

1  if (Serial.available()>0){
2      Serial.println("Hay datos en el buffer");
3  }else{
4      Serial.println("El buffer esta vacío");
5  }

```

4.2.4. Serial.read()

Ya sabemos mostrar la información y si tenemos algo disponible, ahora nos falta poder leer la información y guardarla en una variable, lo cual puede ser logrado con el comando *Serial.read*, que al usarse, debe ser asignado a una variable de tipo *char*, cabe destacar que esta instrucción sólo lee el primer carácter que haya en el buffer, es decir, sólo leerá una letra a la vez.

```

1  char input;
2  if (Serial.available()>0){
3      input = Serial.read()
4      Serial.print("El caracter que leí es: ");
5      Serial.println(input);
6  }

```

4.2.5. Strings y Serial.readStringUntil('\n')

Como pueden suponer, la función anterior puede ser impráctica a la hora de querer leer palabras y/o frases completas de forma rápida y cómoda, por lo cual primero hay que introducir un nuevo tipo de variable, los **String**, que básicamente son variables que almacenan cadenas de caracteres, formando así palabras o frases completas. Funcionan similar a las listas en el sentido que se puede acceder a cada letra de la frase con su índice.

Los strings se pueden crear de la siguiente forma:

```

1  String palabra = "hola mundo";

```

Fijarse que las palabras (o frases) deben estar entre comillas, las cuales pueden ser simples (") o dobles (").

Y se representa de la siguiente manera:

Palabra				
Índice	0	1	2	3
Valor	H	o	l	a

Ahora, para poder leer el string completo desde la consola serial, se necesita el siguiente comando especial: *Serial.readStringUntil('\n')*, el cual sirve para leer muchos caracteres hasta que nosotros le indiquemos, en este caso el *\n*, que representa un salto de línea, el cuál se pone cada vez que apretamos la tecla 'enter'.

Al igual que *Serial.read()*, este comando debe ser asignado a una variable, en este caso, de tipo **String**. Quedando algo así:

```

1  String palabra;
2  if(Serial.available()){
3      palabra = Serial.readStringUntil('\n');
4  }

```

4.3. Debugging

Ahora que tienen todas las herramientas para comunicarse con su placa, existe un concepto en la programación que es el *Debugging*, que básicamente es el proceso para descubrir que es lo que está funcionando mal dentro de nuestro código.

Cuando estemos trabajando en diferentes proyectos de robótica, es probable que no todo funcione correctamente a la primera, es por eso que una técnica básica de *debuggear* es mostrar por pantallas las diferentes variables o mensajes para saber en qué parte del código está el problema, puede ser que nos quedamos atrapados en un bucle sin saber, o que algún cálculo no está bien hecho y no da el resultado esperado, y de esta forma va a ser más fácil descubrir los errores.

4.4. Ejercicios

4.4.1. Preséntate!!

A continuación crea un programa que pregunte por tu nombre y luego lo imprima por la consola serial.

Input:

```

Ingrese su nombre:
> Benjamin

```

Output:

```

Hola benjamin

```

A continuación te mostramos cómo se realiza esto utilizando el último comando que aprendimos, y queda como desafío personal resolverlo sólo con el *Serial.read()*:

```

1  void setup() {
2      //Se inicializa la comunicación serial
3      Serial.begin(9600);
4  }
5
6  void loop() {
7      Serial.println("Ingrese su nombre");
8      //Este while true es para que sólo pregunte una
9      //vez y se quede esperando el input
10     while (true) {
11         if (Serial.available()) {
12             //Se lee la entrada como un string, eso
13             //facilita las cosas
14             String input = Serial.readStringUntil('\n');
15             Serial.print("Hola, ");
16             Serial.println(input);
17             break;
18         }
19     }
20 }

```

4.4.2. Círculos

A continuación crea un programa que pregunte por un radio y devuelva su perímetro y área.

```
Ingrese el radio:  
> 5
```

Output:

```
Perimetro: 31.4  
Area: 78.5
```

El primer paso en el diseño de algoritmos es tener bien claro lo que se pide y ser ordenado al respecto.

En este caso primero tenemos que analizar los datos que tenemos, siendo el radio que es dado por el usuario, y el perímetro y radio, que es lo que nos devuelve el algoritmo como salida, notar que estamos trabajando con decimales, entonces las variables a utilizar son de tipo *float*:

```
1 float radio, area, perimetro;  
2  
3 void setup() {  
4   //Se inicializa la comunicación serial  
5   Serial.begin(9600);  
6 }
```

Luego, hay que pensar en cómo obtener lo que nos piden, y para eso tenemos las fórmulas de perímetro y radio:

$$\text{Perimetro} = 2 * \pi * \text{radio}$$
$$\text{Area} = \pi * \text{radio}^2$$

Lo que traducido en código nos queda así (recordar que $\pi = 3,14$ y $\text{radio}^2 = \text{radio} * \text{radio}$):

```
1 perimetro = 2 * 3.14 * radio  
2 area = 3.14 * radio * radio
```

Y ya con esto podemos armar el resto del código:

```
1  
2 void loop() {  
3   Serial.println("Ingrese el radio");  
4   //Este while true es para que sólo pregunte una  
   vez y se quede esperando el input  
5   while (true) {  
6     if (Serial.available()) {  
7       //Se lee la entrada como un string, eso  
       facilita las cosas  
8       String input = Serial.readStringUntil('\n');  
9       //Se transforma el string a float  
10      radio = input.toFloat();  
11      //Se hacen los cálculos  
12      perimetro = 2 * 3.14 * radio;  
13      area = 3.14 * pow(radio, 2);  
14      //Se imprime el resultado  
15      Serial.print("El perimetro es: ");  
16      Serial.println(perimetro);  
17      Serial.print("El area es: ");  
18      Serial.println(area);  
19      //El break es para salir del while y volver  
      a preguntar el radio  
20      break;  
21    }  
22  }  
23 }
```

Más herramientas Como los *Strings* son palabras, no pueden ser utilizadas en operaciones matemáticas, es por eso que es necesario utilizar la función *.toFloat()* para hacer la transformación.

Más herramientas Arduino ofrece una variedad inmensa de funciones matemáticas, entre ellas *pow()*, el cual toma el primer valor y lo eleva en base al segundo

4.5. Patrones

Si se fijaron, en los 2 ejercicios que hicimos, hay elementos que se repiten e incluso se programan casi igual. Esto es lo que denominamos patrones, y es importante que sean capaces de descubrirlos cuando enfrenten un problema, ya que quizás alguna parte de este ya lo resolvieron anteriormente.

4.6. Desafíos

Ahora te invitamos a intentar resolver los siguientes problemas:

4.6.1. Triángulos

Los tres lados a , b y c de un triángulo deben satisfacer la desigualdad triangular: cada uno de los lados no puede ser más largo que la suma de los otros dos.

$$a < (b + c)$$

$$b < (a + c)$$

$$c < (a + b)$$

Escriba un programa que reciba como entrada los tres lados de un triángulo, e indique:

- Si el triángulo es inválido.
- Si no lo es, qué tipo de triángulo es.

Ejemplo 1:

Input:

```
Ingrese a:
>3.9
Ingrese b:
>6.0
Ingrese c:
>1.2
```

Output:

```
No es un triangulo valido.
```

Ejemplo 2:

Input:

```
Ingrese a:
>1.9
Ingrese b:
>2
Ingrese c:
>2
```

Output:

```
El triángulo es isosceles.
```

Ejemplo 3:

Input:

```
Ingrese a:
>3
Ingrese b:
>5
Ingrese c:
>4
```

Output:

```
El triángulo es escaleno.
```

4.6.2. Secuencia de Collatz

La secuencia de Collatz de un número entero se construye de la siguiente forma:

- si el número es par, se lo divide por dos.
- si es impar, se le multiplica tres y se le suma uno.
- la sucesión termina al llegar a uno.

La conjetura de Collatz afirma que, al partir desde cualquier número, la secuencia siempre llegará a 1. A pesar de ser una afirmación a simple vista muy simple, no se ha podido demostrar si es cierta o no.

Usando computadores, se ha verificado que la sucesión efectivamente llega a 1 partiendo desde cualquier número natural menor que 2^{25}

Desarrolle un programa que entregue la secuencia de Collatz de un número entero.

Ejemplo 1:

Input:

```
n:
>18
```

Output:

```
18 9 28 14 7 22 11 34 17 52 26 13 40 20 10 5
16 8 4 2 1
```

Ejemplo 2:

Input:

```
n:
>19
```

Output:

```
19 58 29 88 44 22 11 34 17 52 26 13 40 20 10 5
16 8 4 2 1
```

Ejemplo 3:

Input:

```
n:
>20
```

Output:

```
20 10 5 16 8 4 2 1
```

Más herramientas Además de los típicos operadores matemáticos (+, -, *, /) existe uno muy útil: %, el cuál da como resultado el resto de una división, por ejemplo:

$$4 \% 2 = 0$$

$$3 \% 2 = 1$$

4.6.3. No múltiplos

Escriba un programa que muestre los números naturales menores o iguales que un número n determinado, que no sean múltiplos ni de 3 ni de 7.

Input:

```
Ingrese numero:  
>24
```

Output:

```
1  
2  
4  
5  
8  
10  
11  
13  
16  
17  
19  
20  
22  
23
```

4.6.4. Alzas del dólar

Un analista financiero lleva un registro del precio del dólar día a día, y desea saber cuál fue la mayor de las alzas en el precio diario a lo largo de ese período.

Escriba un programa que pida al usuario ingresar el número n de días, y luego el precio del dólar para cada uno de los n días.

El programa debe entregar como salida cuál fue la mayor de las alzas de un día para el otro.

Si en ningún día el precio subió, la salida debe decir: **No hubo alzas.**

Input:

```
Cuantos dias?  
>10  
Dia 1:  
>496.96  
Dia 2:  
>499.03  
Dia 3:  
>496.03  
Dia 4:  
>493.27  
Dia 5:  
>488.82  
Dia 6:  
>492.16  
Dia 7:  
>490.32  
Dia 8:  
>490.67  
Dia 9:  
>490.89  
Dia 10:  
>494.10
```

Output:

```
La mayor alza fue de $3.34
```

4.6.5. Promoción con descuento (Difícil)

El supermercado Musta Market ha lanzado una promoción para todos sus clientes. La promoción consiste en aplicar un descuento por cada n productos que pasan por caja.

El primer descuento es de 20 %, y se aplica sobre los primeros n productos ingresados. Luego, cada descuento es la mitad del anterior, y es aplicado sobre los siguientes n productos.

Por ejemplo, si $n = 3$ y la compra es de 11 productos, entonces los tres primeros tienen 20 % de descuento, los tres siguientes 10 %, los tres siguientes 5 %, y los dos últimos no tienen descuento.

Escriba un programa que pida al usuario ingresar n y la cantidad de productos, y luego los precios de cada producto. Al final, el programa debe entregar el precio total, el descuento total y el precio final después de aplicar el descuento.

Input:

```
n:  
>3  
Cantidad productos:  
>8  
Precio producto 1:  
>400  
Precio producto 2:  
>800  
Precio producto 3:  
>500  
Precio producto 4:  
>100  
Precio producto 5:  
>400  
Precio producto 6:  
>300  
Precio producto 7:  
>200  
Precio producto 8:  
>500
```

Output:

```
Total: 3200  
Descuento: 455  
Por pagar: 2745
```

4.6.6. Intersección de círculos(Difícil)

Una circunferencia en el plano está definida por tres valores: las coordenadas (x, y) de su centro, y su radio r .

Escriba un programa que determine si dos circunferencias se intersectan o no.

Input:

```
x1:  
>5  
y1:  
>6  
r1:  
>3.5  
x2:  
>10  
y2:  
>5  
r2:  
>3
```

Output:

```
Se intersectan
```

Input:

```
x1:  
>3.5  
y1:  
>5  
r1:  
>2  
x2:  
>10  
y2:  
>4  
r2:  
>3
```

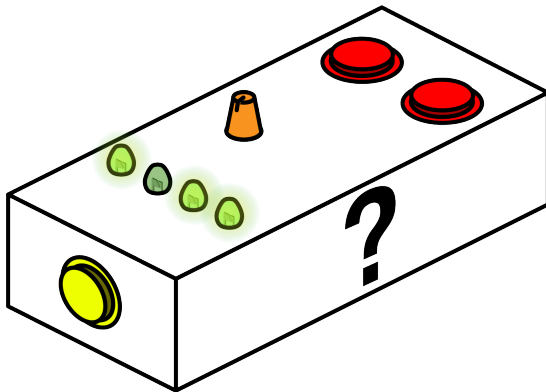
Output:

```
No se intersectan
```

5. Laboratorio de electrónica y programación:

5.1. Ingeniería inversa:

El objetivo de este Laboratorio es construir y programar una copia de un dispositivo con el que sólo podrán interactuar de manera externa, la **CAJA MISTERIOSA**:



Durante el ejercicio, no podrán ver las conexiones internas de la caja misteriosa, ni el código utilizado para hacerla funcionar. Deberán replicar el funcionamiento de esta utilizando **ingeniería inversa**. Los pasos a seguir recomendados para resolver este desafío son:

- **Reconocer:** Utilizar y entender el funcionamiento de la caja, anotar los componentes que creen que contiene y escribir las acciones de cada uno, y como están relacionados entre si.
- **Diagramar:** Con el conocimiento obtenido durante el reconocimiento de la caja, realizar un bosquejo o diagrama de los componentes y las conexiones entre estos y el microcontrolador. Una vez terminado, deberá ser validado por sus mentores para poder continuar con el proceso.
- **Prototipar:** Crear un prototipo utilizando el diagrama validado, conectar los componentes y programar el microcontrolador, intentando replicar el comportamiento observado.
- **Iterar:** Tomar el resultado del prototipo y volver al primer paso para implementar mejoras y acercar el funcionamiento al dispositivo original, comparando el funcionamiento de ambos.

6. Laboratorio extra:

6.1. Calculadora:

El objetivo de este proyecto es construir y programar una calculadora en base a dos componentes:

- Una matriz de botones de 4x4.
- Una pantalla OLED de 1 pulgada

Nuestra calculadora deberá como mínimo:

1. Recibir como entrada un número como una secuencia de dígitos provenientes del teclado.
2. Recibir una operación algebraica (+, -, ·, /).
3. Recibir un segundo número.
4. Entregar el resultado de la operación al apretar el botón asignado a =.

Antes de comenzar, veremos como se utilizan los dos componentes necesarios para la experiencia:

6.1.1. Pantalla OLED

La pantalla OLED la controlaremos utilizando las librerías *Adafruit_GFX.h* y *Adafruit_SSD1306.h*, que nos permitirán escribir texto, figuras e imágenes mediante el uso del protocolo de comunicación I2C. Debemos instalarlas desde el “administrador de librerías” ubicado en la pestaña “Herramientas”.

Primero, debemos conectar la pantalla como se muestra en la siguiente figura, asegurándonos de conectar:

- VCC a VCC
- GND a GND
- SDA a pin 2 (SDA)
- SCL a pin 3 (SCL)

En el código, debemos incluir las librerías necesarias:

```
1 // Incluir librerías encesarías
2 #include <SPI.h>
3 #include <Wire.h>
4 #include <Adafruit_GFX.h>
5 #include <Adafruit_SSD1306.h>
```

Luego, definimos las dimensiones de la pantalla:

```
1 // Definir ancho en pixeles
2 #define ANCHO_OLED 128
3 // Definir ancho en pixeles
4 #define ALTO_OLED 64
```

Para poder configurar las características de la pantalla OLED

```
1 // Asociar el reinicio de la pantalla a arduino
2 #define RESET_OLED -1
3 // inicializar el display
4 Adafruit_SSD1306 display(ANCHO_OLED, ALTO_OLED, &Wire, RESET_OLED);
```

En el *setup()*, inicializamos la comunicacion con la pantalla, y dejamos una advertencia en la comunicacion serial en caso de que falle.

```
1 void setup() {
2     // Iniciar comunicacion serial
3     Serial.begin(9600);
4     // Revisar conexión correcta
5     if(!display.begin(SSD1306_SWITCHCAPVCC, 0x3C))
6     {
7         Serial.println(F("conexión con SSD1306 fallida"));
8         for(;;); // Detener el proceso
9     }
10    // Mostrar imagen por defecto
11    display.display();
12    delay(2000);
13 }
```

Una vez inicializada la pantalla, podemos utilizar cualquiera de las siguientes funciones en el loop para dibujar en la pantalla distintos objetos:

1. **Dibujar pixeles:** Para dibujar pixeles individuales, utilizando la función *drawPixel()*:

```
1 display.drawPixel(x, y, WHITE);
```

2. **Dibujar una linea:** Para dibujar lineas, se puede usar la función *drawLine()*:

```
1 display.drawLine(x1, y1, x2, y2, WHITE)
```

3. **Dibujar una figuras:** podemos dibujar distintas figuras, incluyendo:

- Triangulos:

```
1 display.drawTriangle(10, 10, 55, 20, 5, 40, WHITE);
```

- Rectangulos:

```
1 display.drawRect(x, y, ancho, alto, WHITE)
```

- Circulos:

```
1 display.drawCircle(64, 32, 10, WHITE);
```

Para poder Visualizar lo que dibujamos en la pantalla, se debe realizar el siguiente procedimiento.

1. Borrar todo lo que está dibujado en la pantalla, utilizando la función *clearDisplay()*
2. Dibujar en la pantalla con algunas de las funciones mencionadas.
3. Mostrarlo en pantalla, utilizando la función *display()*

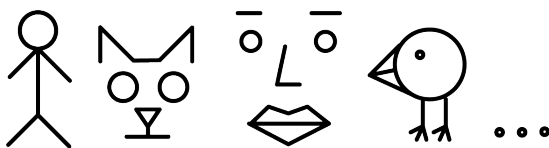
```

1 void loop(){
2   display.clearDisplay();
3   // acá dibujar en la pantalla
4   // con alguna de las funciones
5   display.display();
6   delay(1000);
7 }

```

Ejercicio: Dibujar una pequeña figura en la pantalla para entender las posiciones y la estructura de los píxeles. recomendamos elegir alguno de estos ejemplos:

- Un monito de palo
- Un gato
- Una cara
- Un pollito
- Lo que se te ocurra!



Además de poder dibujar figuras, podemos escribir **texto**, utilizando la función `println()`, para ello, debemos configurar los siguientes parámetros:

1. **Tamaño del texto:** Debemos definir el tamaño del texto que se mostrará en pantalla utilizando la función `setTextSize()`
2. **Color del texto:** Como nuestra pantalla es **monocromática**, esta opción puede ser redundante, pero, podemos de todas maneras configurar el color para que sea blanco, utilizando la función `setTextColor()`
3. **Posición del cursor:** Debemos configurar la posición donde comenzara a escribirse el texto, posicionando el cursor en el píxel (x,y), siendo (0,0) la esquina superior izquierda y (127, 63) la esquina inferior derecha.
4. **Escritura** Finalmente, podemos escribir texto utilizando la función `println()`.

```

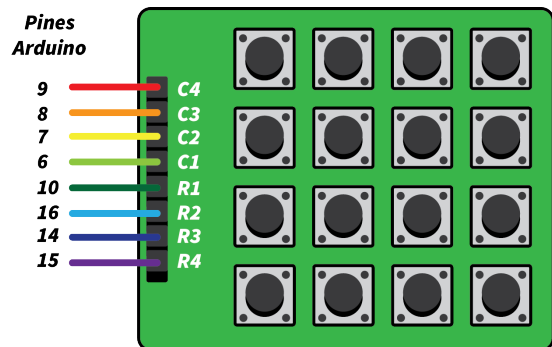
1 void loop(){
2   // Borrar la pantalla
3   display.clearDisplay();
4   // Configurar el tamaño
5   display.setTextSize(2);
6   // Configurar el color
7   display.setTextColor(WHITE);
8   // Posicionar el cursor
9   display.setCursor(10,10);
10  // Escribir
11  display.println("Hola Mundo!");
12  // Mostrar
13  display.display();
14  delay(1000);
15 }

```

Más funciones y ejemplos se encuentran en la documentación <https://learn.adafruit.com/adafruit-gfx-graphics-library/overview> y en el tutorial <https://randomnerdtutorials.com/guide-for-oled-display-with-arduino/>

6.1.2. Matriz de botones 4x4

Tenemos a nuestra disposición una matriz de 16 botones, para leerlos, utilizaremos la librería *Keypad*, para leer progresivamente las columnas y filas que conectan los botones y discernir cuales están siendo apretados dependiendo de por donde pasa corriente. Primero, conectemos los pines correspondientes:



Una vez conectados, podemos realizar la configuración de los botones. Recomendamos el uso de los pines que se muestran en la imagen, para no interferir con los pines **2** y **3** utilizados por la pantalla OLED

En el código, debemos incluir las librerías necesarias:

```

1 // Incluir librerías necesarias
2 #include <Keypad.h>

```

Luego, definimos las cantidad de filas y columnas:

```

1 const int FIL_NUM = 4; //cuatro filas
2 const int COL_NUM = 4; //cuatro columnas

```

Podemos definir los caracteres que nos entregará cada vez que apretamos un botón:

```

1 char keys[FIL_NUM][COL_NUM] = {
2   {'1','2','3','+'},
3   {'4','5','6','-'},
4   {'7','8','9','*'},
5   {'0','.','=','/'},
6 };

```

El siguiente paso es definir los pines a los que se realizaron las conexiones:

```

1 byte pin_filas[FIL_NUM] = {10, 16, 14, 15}; //
   pines conectados a las filas
2 byte pin_columnas[COL_NUM] = {6, 7, 8, 9}; //
   pines conectados a las columnas

```

Definimos el objeto **"keypad"** usando la librería, que nos permitirá leer fácilmente si se está apretando un botón.

```

1 Keypad keypad = Keypad( makeKeymap(keys),
   pin_filas, pin_columnas, FIL_NUM, COL_NUM );

```

Una vez inicializado, solo queda leer la matriz de botones utilizando la función *keypad.getKey()*

```
1 char boton = keypad.getKey();
```

Esta función devolverá el caracter definido en la matriz de caracteres que corresponde al botón apretado o el valor *NO_KEY* si no se ha apretado un botón.

Mas información sobre la librería puede encontrarse en su documentación <https://playground.arduino.cc/Code/Keypad/>.

Referencias

- [1] SparkFun. *What is Electricity?* 2000. URL: <https://learn.sparkfun.com/tutorials/what-is-electricity/all> (visitado 29-01-2023).
- [2] Materials Science & Engineering Student. *Why do Metals Conduct Electricity?* 2023. URL: <https://mstudent.com/why-do-metals-conduct-electricity/> (visitado 04-02-2023).
- [3] Georgia State University. *DC Circuit Water Analogy*. 2005. URL: <http://hyperphysics.phy-astr.gsu.edu/hbase/electric/watcir.html> (visitado 29-01-2023).